



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE
Corso di Laurea Triennale in Informatica

Una metodologia computazionale per il riconoscimento di clustering alternativi nella trascrittomica spaziale

*A computational methodology for the recognition of
alternative clustering in spatial transcriptomics*

CANDIDATO:
Matteo Cerinelli

RELATORE:
Prof. Vincenzo Bonnici

MATRICOLA:
294907

Indice

1	Introduzione	9
2	Concetti generali	11
2.1	Gene	11
2.1.1	Epigenoma	12
2.1.2	Trascrittoma	13
2.2	Trascrittomica spaziale	13
2.2.1	HVGs	14
2.2.2	SVGs	15
2.2.3	Ottenimento dei dati	16
2.2.4	Casi d'uso	17
2.3	Algoritmi per la trascrittomica spaziale	19
2.3.1	Seurat	19
2.3.2	GraphST	20
2.3.3	Giotto	20
2.3.4	GIST	21
2.3.5	Stardust	21
2.4	Limiti	22
3	Nozioni di base	25
3.1	Preprocessing dei dati	27
3.1.1	Smoothing spaziale	27
3.1.2	Partizionamento delle espressioni	29
3.1.3	Creazione maschere binarie	30
3.2	Formato dei dati	31
3.3	Scanpy	32
3.4	Ottenimento Maschere Binarie dei geni	34
4	Sviluppo e risultati sperimentali	43
4.1	Lista degli spot vicini	43
4.2	Divisione in sotto geni	45
4.3	Calcolo del rapporto	50

4.4 Grafo dei rapporti	53
5 Conclusione	55
Bibliografia	57

Elenco delle figure

2.1	HVGs e SVGs a confronto	14
2.2	Tecniche di trascrittomica spaziale basate sulla microdissezione a cattura laser	17
2.3	Applicazione della trascrittomica spaziale nella ricerca sul cancro	19
3.1	Rappresentazione spots (s_i) e geni (i vari colori)	26
3.2	Rappresentazione spaziale spots	27
3.3	Dimostrazione di Smoothing Spaziale	29
3.4	File che contiene la foto del tessuto: <code>tissue_hires_image.png</code> .	32
3.5	Rappresentazione di alcuni geni prima e dopo il filtro di media .	37
3.6	Dimostrazione di come diverse k rappresentano diversamente i dati	37
3.7	Esempio di Knee Point	38
3.8	Gene da dati iniziali a maschera binaria	42
4.1	Rappresentazione grafica degli spot vicini	45
4.2	Maschera binaria di un gene con evidenti sottogeni	46
4.3	Risultato individuazione sottogeni	49
4.4	Due sottogeni VICINI messi a confronto	52

Elenco degli algoritmi

3.1	Funzione per caricare il file .h5 ed esempio su come utilizzarla per inizializzare oggetto AnnData	34
3.2	Funzioni per ottenere i dati di un gene specifico e applicarci il filtro di media	35
3.3	Funzioni per ottenere BIC e KP	38
3.4	Funzioni per ottenere le maschere binarie una volta ottenuta la k del Knee Point	41
4.1	Trasformazione delle coordinate spaziali in coordinate cartesiane	44
4.2	Creazione della lista dei vicini per ogni spot	44
4.3	Identificazione sottogeneri tramite BFS	47
4.4	Creazione completa sottogeneri	49
4.5	Calcolo del rapporto dei sottogeneri	51
4.6	Creazione CSV grafi	53

Elenco delle tabelle

3.1	Estratto metadati spot (dataset 1.DLPFC).	31
3.2	Struttura di <code>AnnData.X</code> e della sua trasposta	36

Capitolo 1

Introduzione

L'introduzione deve contenere un riassunto del lavoro di Tesi. In particolare bisogna esprimere chiaramente e molto sinteticamente: contesto dello studio, motivazioni, contributo e conclusioni. Bisogna quindi fare un sommario dello studio ad alto livello, fornendo le intuizioni senza ricadere in dettagli tecnici. Anche lo stile dovrebbe essere più discorsivo rispetto alle parti tecniche della tesi.

Capitolo 2

Concetti generali

Da sempre tutto ciò che circonda l'uomo è stato fonte di curiosità e studio. Partendo dall'interesse di tutti i vari aspetti riguardanti l'essere umano e gli organismi viventi, con cui ci relazioniamo quotidianamente, si è sviluppata la branca della biologia

L'informatica, invece, la branca della scienza che si occupa dello studio e dell'elaborazione delle informazioni, è stata fin da sempre utilizzata come strumento per migliorare la qualità della vita dell'uomo[1]. Dalla prima calcolatrice meccanica, la *Pascalina*, inventata da Blaise Pascal nel 1644[2], alla *Macchina di Turing*, ideata da Alan Turing nel 1943[3], fino ai moderni computer quantistici, l'informatica ha sempre avuto un ruolo fondamentale nell'evoluzione della società umana.

Lo sviluppo di entrambe queste discipline col passare degli anni ha portato ad un'innovazione: la *bioinformatica*. La bioinformatica nasce dalla combinazione delle due materie prima elencate in quanto gli scienziati manifestavano sempre più l'esigenza di gestire i dati rilevati dalla biologia molecolare, questo perchè, spesso, i dati superano le decine di gigabyte e serviva un modo per poter interpretare più facilmente i campioni raccolti. Oggi la gran parte delle informazioni bioinformatiche possono essere reperite sul *National Center of Biotechnology Information (NCBI)* che fornisce banche dati gratuite e strumenti per la ricerca in biologia molecolare e bioinformatica[4].

2.1 Gene

Ogni organismo, eucariote o procariote, pluricellulare o unicellulare che sia, presenta una molecola fondamentale che contiene tutte le informazioni necessarie per costituire l'organismo, tale molecola è il DNA (acido desossiribonucleico). Il DNA è composto da subunità, chiamate nucleotidi¹, contenenti: un

¹Il nucleotide è l'unità di base che forma DNA e RNA

gruppo fosfato, uno zucchero pentoso e una base azotata. Una specifica sequenza di DNA forma un gene e, l'insieme di tutti i geni, genera il genoma che, oltre a racchiudere le informazioni genetiche, presenta anche le parti del DNA che non codificano proteine². Circa il 30% del DNA nel genoma umano si trova negli esoni e negli introni. I primi, gli esoni, sono sequenze che codificano gli aminoacidi, i quali, vengono espressi in proteine, i secondi, gli introni, invece non codificano nessuna proteina. Al giorno d'oggi sappiamo che il genoma umano presenta circa 20000 geni che codificano proteine[5]. Con il passare degli anni sono stati sequenziati molti genomi, sia di microrganismi che di animali, questo perché, l'interesse degli scienziati è quello di andare definire i processi evolutivi degli esseri viventi, ma, per ottenere dei risultati, è indispensabile andare a comparare numerosi genomi di organismi differenti.

La branca della bioinformatica è molto ampia, per questo, noi ci interesseremo principalmente del genoma. La bioinformatica ha un ruolo fondamentale negli studi della struttura del genoma, per esempio, nell'assemblaggio e annotazione di genomi, o nella caratterizzazione del profilo di espressione genica, generando informazioni cruciali per orientare i successivi studi di genomica funzionale, i quali, si avvalgono di una grande varietà di tecniche sperimentali[6].

Possiamo quindi dire che tutte le cellule di un organismo hanno lo stesso genoma, ma possono presentare epigenomi e trascrittomi molto diversi.

2.1.1 Epigenoma

L'*epigenoma* è una moltitudine di composti chimici e preteine che regolano il genoma su cosa fare senza cambiare la sequenza del DNA. Esso contiene le varie istruzioni per poter costruire le proteine necessarie per svolgere le diverse funzioni della cellula. L'epigenoma, poi, può attaccarsi al DNA e impartire diversi comandi, tra cui l'attivazione o la disattivazione di geni, o il controllo della produzione di determinate proteine all'interno della cellula stessa.

Un essere umano possiede trilioni di cellule distribuite nei tessuti: muscolare, nervoso e osseo e, ciascuna delle cellule che compongono i tessuti, presenta essenzialmente lo stesso genoma all'interno del nucleo³. Le differenze tra le cellule sono determinate da come e quando diversi gruppi di geni vengono attivati o disattivati in vari tipi di cellule. Se prendiamo, ad esempio, in esame le cellule specializzate nell'occhio, queste attivano specifici geni che producono proteine in grado di rilevare la luce, mentre le cellule specializzate nei globuli

²Le proteine sono macromolecole che si formano partendo dal DNA e che vanno a costituire i tessuti.

³il nucleo è una struttura subcellulare, presente negli eucarioti, che contiene il DNA.

rossi producono proteine che trasportano l'ossigeno dall'aria al resto del corpo. L'epigenoma quindi controlla molti di questi cambiamenti nel genoma[7].

2.1.2 Trascrittoma

Il DNA, non viene utilizzato direttamente per costruire le proteine: prima deve essere trascritto in RNA. Questo processo prende il nome di trascrizione e produce diversi tipi di RNA. Il *trascrittoma* rappresenta l'insieme di tutte le molecole di RNA presenti in una cellula in un dato momento.

L'RNA principale è l'RNA messaggero (mRNA) il quale trasporta le istruzioni dai geni ai ribosomi dove verranno generate le proteine mediante l'attività di sintesi proteica. Nei ribosomi, la sequenza di basi dell'mRNA viene letta e tradotta in catene di amminoacidi. Accanto all'mRNA, esistono anche altri tipi di RNA che non codificano proteine ma che svolgono funzioni regolatorie o strutturali, contribuendo a controllare l'attività dei geni e l'organizzazione della cellula. Poiché ogni cellula del corpo umano possiede lo stesso patrimonio genetico, ciò che la distingue dalle altre non è il genoma ma l'insieme dei geni che vengono attivati o silenziati. Questo insieme di trascrizioni geniche costituisce il trascrittoma, e varia notevolmente da cellula a cellula e da tessuto a tessuto. Ad esempio, le cellule muscolari esprimono geni che producono proteine contrattili, mentre le cellule nervose attivano geni che permettono la trasmissione dei segnali elettrici.

Il trascrittoma quindi è lo specchio dell'attività del genoma: mostra quali geni sono effettivamente in funzione in un determinato momento e fornisce informazioni preziose sull'identità, lo stato e la salute delle cellule[7].

2.2 Trascrittomica spaziale

Come detto in precedenza, il trascrittoma rappresenta l'insieme di tutte le molecole di RNA presenti in una cellula in un dato momento. La trascrittomica a singola cellula (scRNA-seq) è diventata essenziale per la ricerca biomedica nell'ultimo decennio, in particolare nello sviluppo della biologia, cancro, immunologia e neuroscienze.

La maggior parte dei protocolli scRNA-seq disponibili in commercio richiedono che le cellule siano recuperate intatte e vitali dai tessuti. Ciò ha precluso lo studio di molti tipi cellulari e ha in gran parte distrutto il contesto spaziale che altrimenti avrebbe potuto informare le analisi dell'identità e della funzione cellulare. Oggi, un numero crescente di piattaforme disponibili in commercio, consente di mappare l'espressione dell'RNA direttamente sulla struttura del tessuto, mantenendo l'informazione di posizione di ciascun segnale: questo approccio è noto come *trascrittomica spaziale (TS)*. La TS produce dati ad alta

dimensionalità utili a descrivere l'organizzazione cellulare e i processi biologici direttamente nel tessuto, preservandone la posizione[8].

Una sfida fondamentale nella TS è l'identificazione degli *Spatially Variable Genes (SVGs)*. Questi ultimi sono geni la cui espressione varia in modo dipendente dalla posizione nello spazio del tessuto. Analizzando lo spazio è possibile scovare pattern non casuali di espressione genica. Identificarli è cruciale nelle analisi di TS perché rivela domini tissutali, le quali sono nicchie cellulari e transizioni; In più facilita l'annotazione⁴ e quindi il clustering[9]. Oltre agli SVGs esistono anche gli *Highly Variable Genes (HVGs)*, geni, che presentano una variabilità di espressione molto superiore a quanto atteso, nonostante ciò non sono in grado di fornire informazioni spaziali rilevanti.

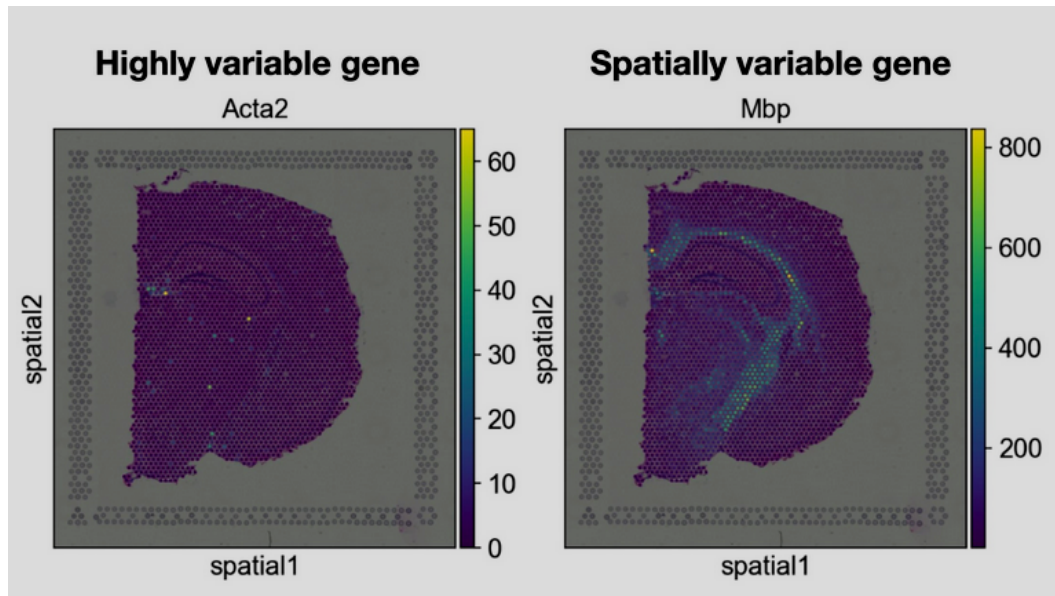


Figura 2.1: HVGs e SVGs a confronto

La figura 2.1 presenta un confronto tra HVGs e SVGs dato lo stesso Dataset. Si noti come gli HVGs non possiedono alcuna informazione spaziale, in quanto, non presentano alcun tipo di pattern; Gli SVGs, al contrario, mostrano una chiara suddivisione in cluster.

2.2.1 HVGs

Come introdotto precedentemente, nell'ambito dell'analisi trascrittomiche, la selezione dei geni più informativi è un passaggio cruciale per ridurre la dimensionalità del dato e concentrare l'attenzione sui segnali biologicamente rilevan-

⁴L'assegnazione di etichette biologiche a cluster o spot

ti. Nel contesto degli HVGs, tali geni sono quelli la cui espressione mostra una variabilità molto maggiore di quella attesa sulla base del rumore tecnico e della media di espressione, e vengono considerati indicatori di eterogeneità biologica[10].

Dal punto di vista formale, in un dataset di espressione genica si valuta la dispersione di ciascun gene, eventualmente corretta per la relazione tra media e varianza che naturalmente sussiste. Le tecniche di identificazione degli HVGs includono modelli statistici, come ad esempio la regressione della varianza stimata sulla media, o trasformazioni che stabilizzano la varianza, con lo scopo di evidenziare quei geni che, rispetto alla media, mostrano una variabilità insolita[11]. Nel caso della trascrittomica spaziale la selezione di HVGs è un passo preliminare molto utile per filtrare i geni che più variano globalmente, in modo da ridurre il rumore e concentrarsi su geni potenzialmente interessanti. Tuttavia, come detto precedentemente, è fondamentale sottolineare che gli HVGs non portano alcuna informazione spaziale: la loro selezione si basa esclusivamente sulla variabilità trascrittomica complessiva, senza considerare la disposizione spaziale degli spot. Questo significa che un gene può essere altamente variabile, in termini di espressione genica, ma non mostrare alcun pattern spaziale. Pertanto, pur essendo utili per filtrare il dataset, gli HVGs da soli non sono sufficienti per definire domini spaziali o per evidenziare nicchie cellulari legate alla morfologia o all'organizzazione tessutale.

Dal punto di vista pratico invece, l'utilizzo degli HVGs usati come base per l'analisi riduce il numero di geni considerati nei passaggi successivi, migliorando l'efficienza computazionale e la stabilità dei modelli. In molti workflow, gli HVGs assumono quindi un ruolo di filtro. Alcuni studi recenti hanno anche confrontato decine di metodi di selezione di HVGs, evidenziando che la scelta del metodo può influire significativamente su tutto il lavoro successivo alla selezione di questi ultimi[12].

2.2.2 SVGs

Gli SVGs sono quei geni la cui espressione genica mostra variazioni dipendenti dalla posizione spaziale all'interno del tessuto: non semplicemente differenze aggregate tra cellule, ma pattern spaziali che riflettono l'architettura del tessuto preso in esame e le nicchie microambientali. L'identificazione degli SVGs è fondamentale perché consente di mettere in luce regioni tissutali con specifici profili molecolari, facilitando la comprensione della morfologia funzionale del tessuto.

Al contrario degli HVGs, gli SVGs implicano che la variabilità rilevante è correlata con la disposizione spaziale degli spot. In altre parole, un gene è SVG se non solo mostra variabilità, ma la sua intensità è anche spazialmente

correlata ai suoi vicini: essi infatti tendono ad avere valori simili, generando così dei pattern. Non tutti gli HVG presentano variabilità spaziale: alcuni potrebbero variare randomicamente nel tessuto, senza rispettare uno schema spaziale rilevante. Viceversa, un gene con espressione moderata può essere un SVG se mostra un pattern spaziale forte e coerente[13].

Selezionare gli SVGs validi non è un compito facile, diversi algoritmi possono produrre set di SVGs molto diversi tra loro nonostante condividano lo stesso dataset. Inoltre, l'accuratezza degli SVGs identificati spesso dipende dalla qualità del clustering dei domini sottostanti: se i domini spaziali sono mal definiti, anche gli SVGs risultanti possono essere meno affidabili. Uno studio ha analizzato e classificato i 14 algoritmi tra i più diffusi utilizzando 60 dataset diversi per cercare di coprire tutte le casistiche possibili. Con questa varietà di dataset è stato possibile testare robustezza, sensibilità, specificità e scalabilità di ciascun metodo in condizioni realistiche. Ma il confronto ha dimostrato che non esiste un metodo universalmente superiore, e che la scelta ottimale dipende dalla natura del dataset, dal livello di rumore, dalla risoluzione spaziale e dalle esigenze analitiche downstream⁵, come il clustering o la ricostruzione delle reti geniche spaziali[14].

2.2.3 Ottenimento dei dati

Per poter recuperare le varie informazioni riguardanti l'espressione genica di un tessuto esistono diversi tipi di approcci[15].

Approcci basati sulla microdissezione mediante cattura laser La microdissezione a cattura laser (*Laser capture microdissection*, *LCM*) consente la dissezione accurata di singole cellule da sezioni di tessuto con notevole precisione, fornendo informazioni spaziali.

Come illustrato nella figura 2.2 il principio di base di questa tecnica è quello di fissare le sezioni di tessuto su vetrini, isolare le aree di interesse dal tessuto utilizzando LCM per poi metterle in soluzioni acquose⁶, quindi catturare l'espressione dei geni nel tessuto isolato tramite sequenziamento⁷. Questo insieme di molecole di DNA preparate appositamente prendono il nome di *library*.

⁵Quello che occorre nel resto della ricerca

⁶Per permettere la reazione enzimatica di ligazione, amplificazione e altre manipolazioni molecolari necessarie nella preparazione della library

⁷Determinare l'ordine dei nucleotidi in molecole di DNA per identificare e quantificare l'RNA in un campione biologico

Analizzando le informazioni sull'espressione genica di ciascuna cellula, o regione separata, è possibile ricostruire la distribuzione spaziale e i profili di espressione genica di singole cellule o regioni all'interno del tessuto.



Figura 2.2: Tecniche di trascrittomica spaziale basate sulla microdissezione a cattura laser

Approcci basati sull'osservazione in situ Gli approcci basati sull'osservazione in situ⁸ rappresentano tecniche che permettono di osservare direttamente la distribuzione di RNA nel tessuto, senza necessità di estrazione preventiva, mantenendo il contesto spaziale originale. I due metodi più comunemente utilizzati sono l'ibridazione in situ (ISH) e il sequenziamento in situ (ISS).

Con l'ISH, vengono preparate molecole, chiamate sonde, di RNA che sono complementari agli RNA bersaglio; Le sonde si legano agli RNA nelle cellule o nel tessuto e sono marcate con molecole fluorescenti, così da poter essere visualizzate al microscopio. Questo processo permette di osservare dove e quanto è espresso un certo RNA. Nonostante ciò, queste tecniche richiedono molteplici cicli di ibridazione e osservazione, e possono essere soggette a rumore⁹.

Con l'ISS si aggiunge un livello in più: l'RNA viene localizzato e ne viene letta la sequenza direttamente nel tessuto.

2.2.4 Casi d'uso

Ad oggi il cancro rimane una sfida significativa per la salute globale. Nonostante i notevoli progressi compiuti negli ultimi decenni, il trattamento del cancro incontra ancora numerosi ostacoli, come la resistenza ai farmaci, la recidiva tumorale e le metastasi. Utilizzando tecniche di sequenziamento trascrittomico ad alto rendimento, le RNA-seq, i ricercatori hanno scoperto numerose alterazioni dell'espressione genica specifiche del cancro in diversi tumori. Tuttavia, il sequenziamento dell'RNA in massa non può ottenere informazioni sull'ambiente tumorale e sull'eterogeneità cellulare[16].

⁸Dal latino: “nel luogo”. Significa analizzare direttamente nel tessuto originale

⁹Forma di disturbo che si aggiunge al segnale utile

La successiva comparsa del sequenziamento dell'RNA a singola cellula, la scRNA-seq, ha inaugurato una nuova era che consente lo studio dei trascrittommi a livello di singole cellule, facilitando analisi approfondite dell'eterogeneità cellulare e l'identificazione di diversi tipi cellulari. Tuttavia, come detto in precedenza, l'esecuzione del sequenziamento a singola cellula richiede che le cellule siano dissociate dal tessuto, causando una perdita di informazioni sulla loro posizione spaziale. Poiché la posizione spaziale delle cellule può effettivamente riflettere l'interazione tra le cellule, ed è strettamente correlata alle funzioni fisiologiche e patologiche, una nuova tecnologia che collega l'espressione genica alle informazioni sulla posizione spaziale è stata molto attesa. La trascrittomicca spaziale rappresenta questa nuova tecnologia, e attraverso il suo utilizzo, è stata ottenuta una comprensione più completa dei meccanismi di inizio e progressione del tumore, offrendo nuove prospettive e opportunità per la diagnosi di malattie e il loro trattamento.

Ad esempio *Beyondcell* è una metodologia progettata per identificare le vulnerabilità ai farmaci utilizzando dati di trascrittomicca spaziale. Viene assegnato un punteggio *Beyondcell* per ogni spot e coppia di farmaci, che va da 0 a 1, il quale indica la suscettibilità al farmaco. Gli spot vengono quindi classificati in cluster e i punteggi vengono visualizzati per comprendere l'efficacia terapeutica del campione[17].

Come già annunciato in precedenza la TS si è quindi rivelata uno strumento rivoluzionario per quanto riguarda la ricerca sul cancro. I tumori, non essendo masse uniformi, sono composti da diverse famiglie cellulari con ruoli ben distinti. Comprendere spazialmente dove ogni cellula si trova e come interagisce con le cellule vicine è stato fondamentale per rivelare i meccanismi di crescita, progressione e resistenza terapeutica del tumore. L'utilizzo di questa tecnologia può facilitare la comprensione dei tipi e degli stati delle cellule che circondano il tumore, nonché delle loro interazioni con le cellule tumorali, rivelando così la complessità del microambiente tumorale (Tumor Microenvironment, TME). Infatti, alcuni studi hanno mappato con la trascrittomicca spaziale i tessuti di vari tumori e hanno confermato il ruolo cruciale del TME nella diagnosi clinica, nella progressione della malattia e nella risposta al trattamento, inclusa l'insorgenza di resistenza al trattamento nei tumori[15].

Il cancro al polmone, ad esempio, è una delle principali cause di morte in ambito tumorale a livello globale ed è una malattia complessa con numerosi sottotipi, ognuno dei quali presenta esiti clinici diversi. Studi su masse tumorali polmonari, hanno rivelato che i macrofagi¹⁰ non si distribuiscono uniformemente: la loro composizione e posizione cambiano, influenzando il comportamento della malattia e la risposta alle terapie. Inoltre, l'integrazione

¹⁰Cellule immunitarie che uccidono batteri, detriti e cellule danneggiate

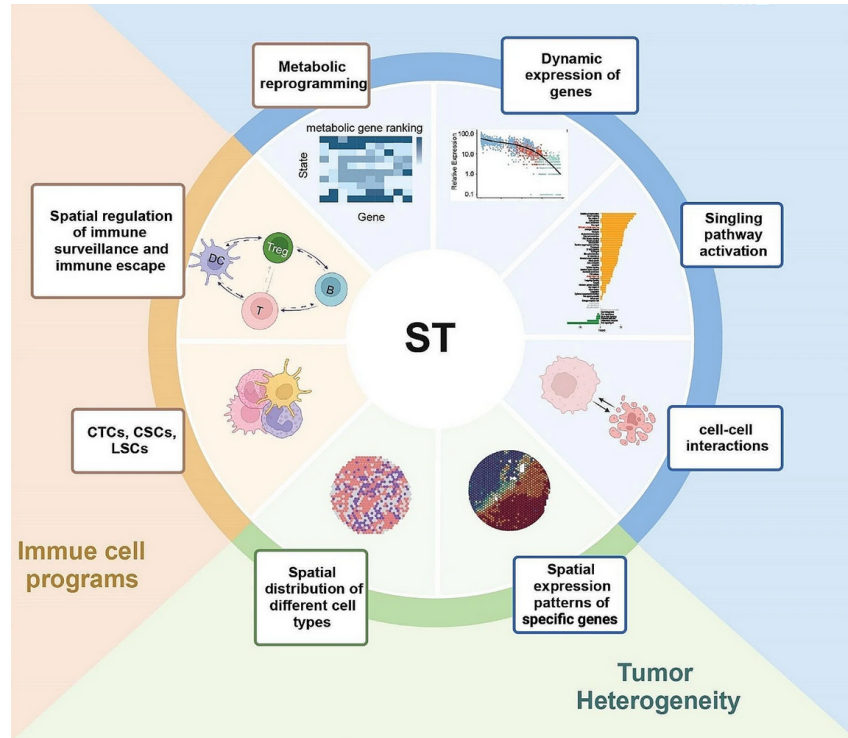


Figura 2.3: Applicazione della trascrittomiche spaziale nella ricerca sul cancro

tra TS e scRNA-seq ha permesso di identificare cellule rare, transizioni cellulari e sottopopolazioni spazialmente specifiche che altrimenti sarebbero rimaste inosservate.

2.3 Algoritmi per la trascrittomiche spaziale

Una volta creata la library dei geni, i dati ottenuti vengono elaborati con algoritmi di analisi spaziale. Tra i più famosi figurano *Seurat*[18], *GraphST*[19], *Giotto*[20], *GIST*[21] e *Stardust*[22].

2.3.1 Seurat

Seurat originariamente era un tool per l'analisi di dati di scRNA-seq, ma da recenti aggiornamenti possiede strumenti utili all'analisi di dati spaziali. Permette funzioni di normalizzazione, identificazione di HVGs e di SVGs, analisi multisezione ed altro. Offre anche funzioni per visualizzare sull'immagine tissutale l'espressione dei geni, cluster e l'integrazione fra slices, anche con opzioni per evidenziare contorni cellulari o centroidi.

In sintesi, Seurat, è un tool ben consolidato e ampiamente usato, con buone interfacce e integrazioni con l’ecosistema scRNA-seq. Tuttavia, non essendo nato per questo scopo, ma essendo stato adattato in seguito, può risultare meno potente di pipeline specializzate.

2.3.2 GraphST

GraphST è un pacchetto più recente, progettato specificamente per dati di trascrittoma spaziale, che sfrutta reti neurali e addestramento self-supervised per integrare informazione spaziale ed espressione genica. Questo algoritmo lavora attraverso l’utilizzo di grafi non diretti, in cui ogni nodo è uno spot diverso, e gli archi collegano gli spot vicini tra loro in base alle coordinate spaziali. Utilizza poi una rete neurale (Graph Neural Network) per apprendere rappresentazioni informative e discriminative negli spot e minimizzandone la distanza dove sono spazialmente adiacenti e viceversa.

Al contrario di Seurat, quindi, GraphST è stato progettato specificamente per dati spaziali, riuscendo ad offrire buone prestazioni rispetto ai competitor. Nonostante ciò, richiede diversa attenzione per la costruzione del grafo, ed avendo una rete neurale richiede più risorse computazionali e tempo per l’addestramento¹¹ rispetto ad altre soluzioni.

2.3.3 Giotto

Giotto è un toolkit open-source scritto in R pensato per l’analisi integrata di dati spaziali, con una serie di moduli per clustering spaziale, rilevamento di domini e interazioni tra cellule. Costruisce internamente una “spatial neighborhood network” che connette spot vicini in base alle coordinate fisiche. Il grafo ottenuto viene poi utilizzato per tutte le analisi spaziali. Questo toolkit offre vari metodi di clustering: sia classici basati su espressione, come l’algoritmo k-means, e sia metodi che integrano la struttura spaziale, in particolare un modello basato su Hidden Markov Random Fields (HMRF) che dà vita a cluster spazialmente coerenti.

Riassumendo Giotto è un toolkit abbastanza completo con molti moduli, incluso il modello HMRF che dà una componente spaziale ben fondata per il clustering. Tuttavia non avendo una rete neurale, al contrario di GraphST, potrebbe non cogliere pattern spaziali complessi.

¹¹Processo di allenamento di un modello per eseguire un compito specifico

2.3.4 GIST

GIST (Graph Information for Spatial Transcriptomics) rappresenta un avanzamento significativo nel campo della trascrittomica spaziale, proponendo un approccio basato su reti neurali (le stesse usate da GraphSt2.3.2) per migliorare l'identificazione dei domini spaziali all'interno di tessuti biologici. Nella TS la corretta delimitazione dei domini spaziali rimane una sfida computazionale complessa: nonostante i notevoli progressi ottenuti, manca ancora un vero e proprio standard di riferimento universalmente riconosciuto.

In tale contesto, GIST propone una soluzione innovativa che combina i dati di espressione genica con le coordinate spaziali, costruendo una rappresentazione grafica del tessuto che riflette la sua struttura biologica reale. Il modello sfrutta le potenzialità delle reti neurali per modellare in modo esplicito le dipendenze spaziali tra gli spot, utilizzando un approccio di apprendimento contrastivo¹² per ottimizzare le rappresentazioni dei nodi nel grafo. Questo consente di catturare con maggiore accuratezza le relazioni locali e globali tra le regioni del tessuto, migliorando in modo significativo l'identificazione dei domini.

Dal punto di vista delle prestazioni, GIST mostra risultati migliori rispetto ai metodi esistenti in termini di metriche di clustering, come l'*Adjusted Rand Index*¹³ (ARI), evidenziando la sua capacità di riprodurre in modo più fedele la struttura intrinseca dei dati trascrittomici spaziali. Inoltre, gli autori introducono una nuova metrica di valutazione, la *Silhouette Spatial Score* (SSS), una estensione della Silhouette Score¹⁴, che incorpora l'informazione spaziale degli spot vicini per valutare simultaneamente la coerenza trascrittomica e la contiguità spaziale dei domini individuati.

Nel complesso, GIST si distingue per la capacità di identificare domini che non solo risultano avere espressione genica simile, ma risultano anche coerenti dal punto di vista spaziale, offrendo così un modello robusto per l'analisi dei tessuti.

2.3.5 Stardust

Stardust è basato sull'algoritmo Seurat e ha come obiettivo quello di migliorare l'analisi dei dati di trascrittomica spaziale concentrandosi in particolare sulla fase di clustering, ovvero sulla suddivisione degli spot del tessuto in domini omogenei. L'innovazione principale di Stardust consiste nell'integrare in modo flessibile le informazioni di espressione genica con quelle spaziali, consentendo

¹²Il modello impara confrontando coppie di esempi

¹³Indica quanto bene sono stati raggruppati gli elementi dato un riferimento

¹⁴Indica quanto bene sono stati raggruppati gli elementi in base ai dati disponibili

all'utente di modulare quanto peso dare allo spazio nella definizione delle similarità tra gli spot. In alternativa, ne esiste anche una versione automatica, Stardust*, che calibra dinamicamente il contributo spaziale basandosi sulla distribuzione delle distanze di espressione.

Come misura principale di valutazione viene adottato il *Cell Stability Score* (CSS), che quantifica la stabilità¹⁵ degli spot e il coefficiente di variazione della distribuzione dei CSS, dove un valore più basso indica performance più stabili e omogenee. Inoltre, per valutare la coerenza dei cluster ottenuti, viene calcolato l'indice di autocorrelazione spaziale *Moran's I* per ogni gene nei cluster, verificando che i geni con elevata autocorrelazione fossero localizzati all'interno delle stesse regioni spaziali.

Moran's I viene definito come segue:

$$I = \frac{n}{W} \cdot \frac{\sum_{i=1}^n \sum_{j=i}^n w_{i,j} z_i z_j}{\sum_{i=1}^n z_i^2} \quad (2.1)$$

dove z_i indica la deviazione del gene dalla media, $w_{i,j}$ è il peso spaziale tra gli spots, n indica il numero di spots e infine W indica la somma di tutti i $w_{i,j}$.

I risultati mostrano che, in molti casi, l'inclusione dell'informazione spaziale tramite Stardust o Stardust* migliora significativamente la stabilità del clustering rispetto a metodi che non la considerano, e raggiunge prestazioni competitive rispetto alle altre tecniche che già sfruttano aspetti spaziali. In sintesi, Stardust propone un compromesso fra informazione trascrittomica e informazione spaziale nel clustering dei dati di trascrittomica spaziale, offrendo uno strumento relativamente semplice da usare.

2.4 Limiti

Nonostante i grandi progressi, la trascrittomica spaziale presenta ancora vari problemi che ne frenano l'adozione su larga scala, tra cui: costi elevati, bassa risoluzione spaziale, lunghi tempi di risposta e aree di cattura ridotte. Inoltre, come detto precedentemente, idealmente gli identificatori molecolari univoci (UMI) in uno spot misurano l'espressione specifica dello spot, ma, spesso, questo non avviene nella pratica a causa della contaminazione causata dal sanguinamento degli spot vicini. Questo fenomeno genera un artefatto chiamato spot swapping. Ignorare tale problematica può portare a risultati fuorvianti, come l'identificazione errata di SVGs o la sovrastima dell'espressione genica in regioni specifiche del tessuto.

¹⁵tendenza che uno spot mantenga il proprio clustering anche sotto perturbazioni del dataset

Nonostante ci siano strumenti specializzati per la risoluzione delle problematiche sopracitate, come SpotClean[23] per rimuovere lo spot swapping, e iSCALE[24] per poter utilizzare tessuti di grosse dimensioni, non esiste ad oggi un algoritmo unico che possa risolvere tutti i problemi della TS. Vari benchmark e review scientifiche hanno portato alla stessa conclusione: occorre una pipeline modulare, che si possa adattare alle esigenze specifiche di ciascun tessuto.

La TS resta un campo molto recente, e nuove analisi, metodi e software, vengono pubblicati ogni mese, perciò non sono da escludere eventuali sviluppi futuri che miglioreranno tale problematica[25].

Capitolo 3

Nozioni di base

Prendendo in esame un tessuto qualsiasi, questo viene sezionato e diviso in celle più piccole, dette *spot*. Uno spot, quindi, non è altro che una posizione pre-definita su una griglia, con un *barcode*¹ che lo identifica. In genere ogni spot ha un diametro di circa $55\text{ }\mu\text{m}$ e con una distanza da spot a spot di circa $100\text{ }\mu\text{m}$ [26].

Possiamo definire l'insieme dei nostri spots nel seguente modo:

$$\mathbb{S} = \{s_1, s_2, \dots, s_n\} \quad (3.1)$$

dove ogni $\mathbb{S}_i$ è uno spot.

Ogni spot, al suo interno, contiene più cellule. Per questo motivo, ogni spot viene associato ad un vettore di espressione genica che dà vita alla matrice $X \in \mathbb{R}^{n \times m}$ dove n è il numero di spot e m il numero di geni. Possiamo definire l'insieme dei nostri geni nel seguente modo:

$$\mathbb{G} = \{g_1, g_2, \dots, g_m\} \quad (3.2)$$

dove ogni g_i è un gene. Di conseguenza si può definire lo spot $\mathbb{S}_i \in \mathbb{G}^m$ come e assegniamo ad ogni $\mathbb{S}_i$ il seguente vettore:

$$(x_{i,1}, x_{i,2}, \dots, x_{i,m}) \quad (3.3)$$

dove $x_{i,j} \in \mathbb{R}$ rappresenta l'espressione del gene g_j allo spot s_i , con $1 \leq i \leq n$.

La figura 3.1 mostra una rappresentazione grafica degli spot e dei geni all'interno di un tessuto. Ogni cella rappresenta uno spot, mentre ogni colore rappresenta un gene diverso². Si può notare che ogni spot $\mathbb{S}_i$, al suo interno, può contenere più geni. Inoltre, nella figura d'esempio, si hanno 36 spot diversi

¹Una breve sequenza di DNA univoca per ogni spot

²Si noti che il colore non è rappresentativo della presenza o meno di quel gene in quello spot

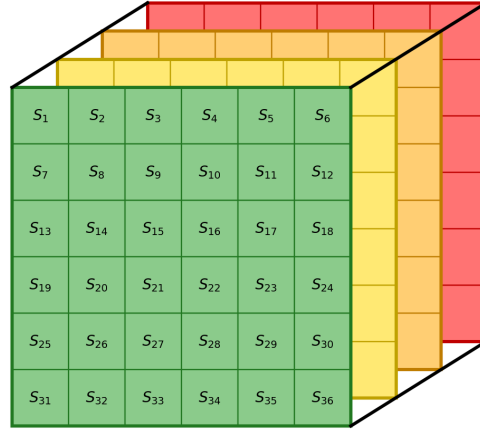


Figura 3.1: Rappresentazione spots (s_i) e geni (i vari colori)

e 4 geni, ma ma nella realtà i dati sono differenti: gli spot possono arrivare a oltre 3000 celle e i geni possono superare i 30,000.

La figura 3.1, tuttavia, presenta una inesattezza in quanto gli spot non sono disposti su una matrice $n \times n$, bensì hanno una rappresentazione esagonale. Questo significa che, dato uno spot s_i , questo presenterà sei spot adiacenti, tranne nei casi in cui gli spot sono localizzati ai bordi del tessuto.

Per poter trovare spazialmente uno spot s_i , quindi, occorre assegnare una coordinata x e una coordinata y ad ogni spot, e, una volta creata una matrice $[s_i, \langle x_i, y_i \rangle]$, dove $\langle x, y \rangle$ indicano le coordinate dello spot, è possibile trovare i 6 vicini attraverso un semplice calcolo. Inoltre, per sopperire ad eventuali spazi vuoti nel tessuto, viene calcolata la distanza euclidea tra gli spot, in modo da poter scartare gli spot che altrimenti verrebbero erroneamente segnati come vicini.

La figura 3.2 mostra una rappresentazione grafica degli spot in un tessuto, con le loro coordinate x e y . Lo spot preso in esempio, come riferimento, è lo spot con coordinate $x = 2, y = 3$.

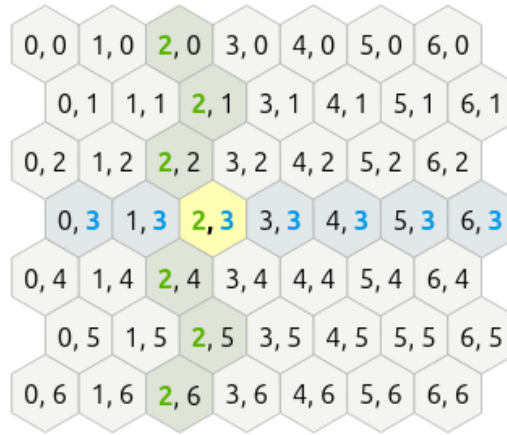


Figura 3.2: Rappresentazione spaziale spots

3.1 Preprocessing dei dati

Quando si entra in possesso di una library, contenente i barcode degli spot, le coordinate spaziali e i relativi metadati, non è consigliato avviare l'analisi dei dati fin da subito, questo perchè i dati di TS contengono anche rumore tecnico e biologico che può mascherare dei pattern in realtà utili, come ad esempio dei geni con profili irregolari. Occorre quindi prima di tutto ripulire e ridurre la complessità dei dati prima di poterli analizzare.

Per lo studio che questa tesi si è posta come obiettivo, il preprocessing consiste in tre fasi principali:

1. Smoothing Spaziale
2. Partizionamento delle espressioni
3. Creazione maschere binarie

Queste 3 fasi sono fondamentali per ottenere per ciascun gene una lista dei vari spot dove il gene ha a tutti gli effetti una sua espressione e dove invece quest'ultima non è presente[27].

3.1.1 Smoothing spaziale

Il primo step, ampiamente utilizzato in gran parte degli algoritmi di trascrittomico spaziale[20, 28, 29], per poter minimizzare il rumore è applicare un *filtro di media* (Mean Filter), ovvero una tecnica per poter sostituire ogni elemento, con la media dei valori dei suoi vicini dato un raggio prestabilito. Questo serve per poter ridurre il rumore locale ed evitare variazioni importanti.

Considerando un tessuto a due dimensioni, contenente S spots e G geni. Possiamo denotare con $y_{ij} \in \mathbb{R}$ il livello di espressione genica di un determinato gene g_i nello spot S_i . Dalla definizione di matrice di espressione genica X definita precedentemente, possiamo ora definire $\hat{X}$ la matrice ottenuta come output dopo aver applicato la seguente operazione:

$$\hat{y}_{ij} = \sum_{s_k \in \mathcal{N}_r(s_j)} w_{jk} \cdot y_{ik} \quad (3.4)$$

dove $\mathcal{N}_r(s_j) = \{s_k \mid \|s_k - s_j\| \leq r, s_k \in S\}$ con raggio $r \in \mathbb{R}$ denota i vicini dello spot s_j , e w_{jk} sono i pesi spaziali che soddisfano $\sum_k w_{jk} = 1$.

Per trovare i pesi spaziali poi, viene applicato il filtro di media:

$$w_{jk} = \frac{1}{|\mathcal{N}_r(s_j)|} \quad (3.5)$$

dove $\mathcal{N}_r(s_j)$ denota i vicini dello spot s_j entro il raggio r , e tutti i vicini contribuiscono in modo equivalente.

In alternativa al filtro di media è possibile utilizzare la *Gaussian Kernel Smoothing*[30] per calcolare i pesi spaziali. Prima si devono calcolare i pesi non normalizzati usando un Kernel Gaussiano:

$$\tilde{w}_{jk} = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\|s_k - s_j\|^2}{2\sigma^2}\right) \quad (3.6)$$

dove σ controlla la portata spaziale di smoothing. Dopodichè andremo ad applicare una normalizzazione³ per ottenere i pesi finali:

$$w_{jk} = \frac{\tilde{w}_{jk}}{\sum_h \tilde{w}_{jh}} \quad (3.7)$$

garantendo che $\sum_k w_{jk} = 1$ per ogni spot s_j .

La figura 3.3 dimostra come filtro di media attenua le variazioni locali, riducendo i valori e producendo una mappa più omogenea dove i picchi isolati di rumore risultano smussati.

Smoothing Spaziale a due fattori Un'altra variante allo smoothing visto sopra è lo *smoothing a due fattori*, che combina i vicini spaziali e la loro somiglianza trascrizionale, ovvero i pattern. Usando lo smoothing a due fattori, i dati smussati risultano essere più accurati nelle loro partizioni e interpretazioni biologiche rispetto all'utilizzo di operazioni a un fattore[31].

³Dividere tutti i termini di un'espressione per uno stesso fattore in modo che l'espressione risultante abbia una certa norma uguale a 1.

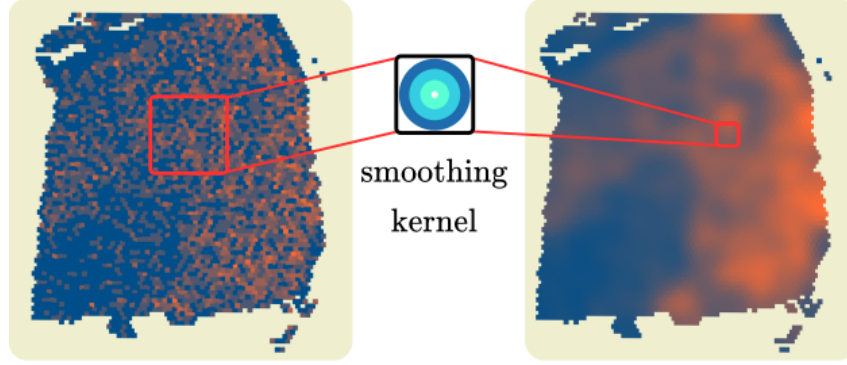


Figura 3.3: Dimostrazione di Smoothing Spaziale

Dato $\mathcal{X}_i$ un vettore di espressione genica per uno spot i , la sua versione smussata $\mathcal{X}'_i$ può essere calcolata come segue:

$$\mathcal{X}'_i = (1 - \alpha) \mathcal{X}_i + \alpha \left(\beta \frac{\sum_{j \in N_s(i)} c_{ji}^s \mathcal{X}_j}{\sum_{k \in N_s(i)} c_{ki}^s} + (1 - \beta) \frac{\sum_{j \in N_p(i)} c_{ji}^p \mathcal{X}_j}{\sum_{k \in N_p(i)} c_{ki}^p} \right). \quad (3.8)$$

Nella equazione 3.8 ci sono due parametri, α e β . La prima regolarizza il rapporto tra l'espressione originale e quella corretta, evitando che l'espressione dello spot in esame risulti eccessivamente smussata. β invece modula il peso relativo dello smussamento basato sulla vicinanza spaziale rispetto a quello basato sui pattern. $N_s(i)$ e $N_p(i)$ indicano, rispettivamente, i vicini spaziali e i pattern nello spot i . c_{ji}^p e c_{ji}^s indicano, rispettivamente, il contributo spaziale e quello di pattern dello spot j vicino allo spot in oggetto i .

3.1.2 Partizionamento delle espressioni

Una volta ottenuti i dati dei vari geni smussati $\hat{y}_i = (\hat{y}_{i1}, \dots, \hat{y}_{in})$ dal filtro di media 3.5, il passo successivo consiste nel suddividere le posizioni spaziali associate a ciascun gene g_i in sotto-regioni omogenee.

Formalmente per ogni gene $g_i \in \mathbb{G}$:

1. I vari valori di espressione genica smussati $\{\hat{y}_{ij}\}_{j=1}^n$ vengono trattati come osservazioni all'interno di uno spazio probabilistico.
2. Su tali valori viene applicato un modello di *Gaussian Mixture Model* (GMM) con k componenti, che consente di rappresentare la distribuzione dei dati come combinazioni di più distribuzioni gaussiane. Il numero di componenti k del modello non viene scelto a priori, ma determinato in modo automatico secondo la seguente procedura:

- (a) Per prima cosa vengono addestrati diversi GMM con un numero crescente di componenti, da 1 a $K_{\max}$, in modo da ottenere un set di modelli ordinati per numero di componenti:

$$\mathbb{M}_i = (GMM_1(\hat{y}_i), GMM_2(\hat{y}_i), \dots, GMM_{K_{\max}}(\hat{y}_i)) \quad (3.9)$$

- (b) Ciascun modello viene poi valutato con una funzione $V : GMM \rightarrow \mathbb{R}$, (come ad esempio AIC, BIC, ecc...), in modo da ottenere il vettore:

$$\mathbb{V}_i = (V(GMM_1(\hat{y}_i)), V(GMM_2(\hat{y}_i)), \dots, V(GMM_{K_{\max}}(\hat{y}_i))) \quad (3.10)$$

- (c) Infine i valori ottenuti vengono rappresentati graficamente e il modello ottimale viene scelto nel *Knee Point*, cioè dove l'incremento di qualità del modello diventa marginale. Questa scelta evita il rischio di *overfitting*⁴, garantendo al contempo una suddivisione coerente delle regioni di espressione.
3. Una volta determinato il modello ottimale, ogni spot s_j viene assegnato alla componente gaussiana con la massima probabilità a posteriori, generando così un insieme di partizioni spaziali

$$\mathbb{P}_i = \{P_{i1}, \dots, P_{ik}\} \quad (3.11)$$

dove ciascun sottogene P_{im} è definito nel seguente modo:

$$P_{im} = \left\{ s_j \mid m = \arg \max_{\ell} P(\ell \mid \hat{y}_{ij}) \right\} \quad (3.12)$$

3.1.3 Creazione maschere binarie

Per ogni gene $g_i \in \mathbb{G}$, generiamo dalle partizioni ottenute la media dell'espressione smussata all'interno di ciascuna regione P_{im} . Dopodichè la regione con la media più bassa viene identificata come zona a bassa espressione:

$$P_i^{\min} = \arg \min_{P \in P_i} \left(\frac{1}{|P|} \sum_{s_j \in P} \hat{y}_{ij} \right) \quad (3.13)$$

Si definisce quindi una maschera binaria $\bar{\bar{y}}_i \in \{0, 1\}^n$ dove:

$$\bar{\bar{y}}_{ij} = \begin{cases} 0, & \text{se } s_j \in P_i^{\min} \quad (\text{bassa espressione}) \\ 1, & \text{altrimenti.} \end{cases} \quad (3.14)$$

⁴Fenomeno per cui un modello si adatta eccessivamente ai dati di training, perdendo capacità di generalizzazione

Questa maschera sintetizza il comportamento spaziale del gene, distinguendo le regioni di minima attività a cui è stato assegnato valore 0, da quelle di maggiore espressione a cui è stato assegnato valore 1. Le maschere così generate costituiscono la rappresentazione compatta e comparabile dei pattern spaziali di ciascun gene.

3.2 Formato dei dati

I dati del campione di tessuto fornitoci sono in formato *10x Visium*, una piattaforma di trascrittomica spaziale che misura l'espressione genica su sezioni di tessuto, mantenendo per ogni misura la posizione spaziale. La cattura avviene su spot barcodati stampati su una slide. Il programma che processa i dati in modo da mappare i barcode con la loro posizione spaziale viene chiamato *Space Ranger* e produce la matrice *Features* $\times$ *Barcodes*, dove con *Features* si intendono i geni, e con *Barcodes* i vari spot; Ogni spot raccoglie RNA da più cellule e i conteggi vengono poi mappati sull'immagine.

I file 10x Visium sono in formato **.h5**, un file specifico pronto per essere elaborato da **ScanPy** una libreria Python⁵ open-source⁶ progettata per l'analisi di dati di trascrittomica. Scanpy infatti offre la funzione `sc.read_visium()` per importare direttamente le cartelle di output di Space Ranger.

Prendendo il file **.h5** e traducendolo in un file **.tsv** (Tab-Separated Values) si può osservare una struttura di questo tipo:

Tabella 3.1: Estratto metadati spot (dataset 1.DLPFC).

Barcode	Row	Col	Cluster	sum_Gene
AAACAAGTATCTCCCA-1	1	3	7	3586
AAACAATCTACTAGCA-1	1	27	4	1150
AAACACCAATAACTGC-1	1	59	8	1960

barcode: identificatore dello spot; **sample_name**: nome del campione;

row, col: coordinate dello spot nella griglia;

Cluster: cluster di appartenenza; **sum_gene**: numero totale di geni rilevati.

La tabella 3.1 è solo una sezione di cosa contiene davvero il file **.h5**, nella realtà sono presenti svariati gigabyte di dati.

Oltre ai file sopracitati viene fornita anche una cartella chiamata **spatial** contenente immagini ad alta e bassa definizione del tessuto, e il file delle posizioni degli spot chiamato **tissue_positions.csv**

⁵Linguaggio di programmazione di alto livello, interpretato e open source

⁶Codice sorgente libero



Figura 3.4: File che contiene la foto del tessuto: `tissue_hires_image.png`

3.3 Scanpy

Scanpy è un toolkit in Python per l'analisi di dati di espressione genica a singola cellula, progettato per lavorare anche con dataset molto estesi e per integrarsi con l'ecosistema Python. Include metodi per il pre-processing, come filtri su cellule e geni, normalizzazione, identificazione degli HVGs e clustering.

Scanpy fornisce inoltre funzioni con velocità elevate che rendono l'analisi di data set con più di un milione di cellule e interrogazioni real time nell'ordine di secondi per data set con circa 100,000 cellule. Questa moltitudine di funzioni, assieme ad velocità non indifferente, ne hanno favorito l'adozione in molti ambiti di ricerca.[32]

Dal punto di vista della manutenibilità invece, Scanpy è sviluppato nell'ambito del progetto *scverse*, che coordina strumenti fondamentali per l'analisi omica in Python. Questo garantisce linee guida comuni, interoperabilità tra pacchetti e una documentazione unificata, elementi cruciali in contesti accademici e clinici dove la tracciabilità delle analisi è essenziale.[33]

Le funzioni della libreria sono organizzate per famiglie:

- `sc.pp.*` riguarda le varie funzioni per il pre-processing.
- `sc.tl.*` serve per gli strumenti analitici, come clustering o creazione di grafi.
- `sc.pl.*` infine riguarda le varie funzioni usate per la visualizzazione, con cui è possibile stampare a video delle immagini.

Per la trascrittoma spaziale, Scanpy offre un'integrazione naturale con i dati 10x Visium tramite `scanpy.read_visium()`, che carica non solo la matrice di espressione ma anche immagini e coordinate degli spot. La funzione ha la seguente firma:

```
scanpy.read_visium(  
    path,  
    genome = None,  
    count_file = 'filtered_feature_bc_matrix.h5',  
    library_id = None,  
    load_images = True,  
    source_image_path = None  
)
```

Gli argomenti della funzione sono i seguenti:

- `path`: Percorso alla cartella radice che contiene l'output Visium, ovvero il file `.h5` e la sottocartella `spatial/`.
- `genome`: Opzionale, è l'identificatore del genoma che si vuole filtrare, se presente nei metadati 10x.
- `count_file`: Nome del file di 10x Visium contenente i vari dati. Deve essere presente nella cartella inserita in `path`. Di default ha come nome (`filtered_feature_bc_matrix.h5` per default).
- `library_id`: Opzionale, è l'identificativo della libreria Visium. Risulta utile quando occorre concatenare più librerie.
- `load_images`: Se `True`, carica le immagini del tessuto.
- `source_image_path`: Opzionale, è il percorso alternativo da cui leggere le immagini se non si trovano in `spatial/`.

La funzione restituisce un oggetto `AnnData`, una struttura dati che accoppia in modo coerente la matrice di espressione con annotazioni su spot e geni, oltre che a campi dedicati per grafi e metadati. `anndata.X` contiene la matrice *Features* $\times$ *Barcodes*, `anndata.obs` da *Observations* contiene tutti e solo i barcodes degli spot, mentre `anndata.vars` da *Variables* contiene tutti e solo i geni misurati, con nome, id e tipo. Gli oggetti `AnnData` si possono salvare tramite la funzione `anndata.write_h5ad()` e i file così creati hanno estensione `.h5ad`.

La funzione sopracitata ha la seguente firma:

```
anndata.write_h5ad(
    filename,
    compression = hdf5plugin.FILTERS["zstd"]
)
```

Con come argomenti:

- `filename`: Il nome del file che si vuole salvare con estensione `.h5ad`.
- `compression`: Opzionale, metodi alternativi di compressione del file.

Un esempio completo di un programma scritto in linguaggio Python con lettura dei dati 10x Visium attraverso Scanpy e relativo salvataggio di `AnnData` in formato `.h5ad` è il seguente:

```
1 import scanpy as sc
2
3 def load_data():
4
5     adata_gene_expression = sc.read_visium("1.DLPFC
6     /151673", count_file="filtered_feature_bc_matrix.h5",
7     load_images=True)
8
9     return adata_gene_expression
10
11 annData = load_data()
12
13 annData.write_h5ad(filename="processed_adata")
```

Codice 3.1: Funzione per caricare il file `.h5` ed esempio su come utilizzarla per inizializzare oggetto `AnnData`

3.4 Ottenimento Maschere Binarie dei geni

Una volta ottenuti i dati in formato `AnnData` è possibile iniziare a manipolarli come oggetto per poterne ricavare le varie informazioni necessarie allo studio che si sta svolgendo.

Come detto in precedenza occorre applicare prima di tutto uno Smoothing per poter ripulire e ridurre la complessità dei dati ottenuti. Questo smoothing viene applicato gene per gene attraverso il filtro di media, definito precedentemente, in questo modo:

```

1  ITNI = np.zeros((nspot, 7), dtype=int)
2  VNI   = (ITNI != -1).astype(int)
3  NN    = VNI.sum(axis=1)
4  NM    = (VNI.T / NN.T).T
5
6  def get_gene_data(data, gene_id):
7      array_data = data.X.toarray().T
8      gene_data  = array_data[gene_id]
9      return gene_data
10
11 def super_opt_mean_filter(gene_data):
12     global ITNI, NM
13     return (gene_data[ITNI] * NM).sum(axis=1)
14
15 def opt_mean_filter_iterated(gene_data, it):
16     for _ in range(it):
17         gene_data = super_opt_mean_filter(gene_data)
18     return gene_data
19
20 geneData = get_gene_data(anndata, 1)
21
22 geneData = opt_mean_filter_iterated(gdata, 5)

```

Codice 3.2: Funzioni per ottenere i dati di un gene specifico e applicarci il filtro di media

Lo Snippet del codice 3.2 presenta 3 funzioni principali che permettono di utilizzare il filtro di media: la prima, `get_gene_data(data, gene_id)`, permette di recuperare dall'oggetto `AnnData` un gene dato il suo id. Il suo funzionamento è molto basilico in quanto intuitivamente viene creata una variabile contenente la matrice trasposta di `AnnData.X`, che, come detto precedentemente, l'oggetto `X` all'interno di `AnnData` contiene la matrice *Features* $\times$ *Barcodes*, e rendendola trasposta essa viene trasformata in *Barcodes* $\times$ *Features*, ovvero anzichè avere in ogni riga uno spot e per ogni colonna l'espressione genica di un determinato gene su quello spot, si otterrà invece un gene per ogni riga, e la sua espressione genica su ogni spot per ogni colonna. La tabella 3.2 contiene una rappresentazione visiva della matrice `AnnData.X` e della sua relativa trasposta.

La seconda funzione, `super_opt_mean_filter(gene_data)`, implementa un filtro di media sui valori di espressione genica sfruttando operazioni completa-

Tabella 3.2: Struttura di `AnnData.X` e della sua trasposta

Matrice $X = \text{AnnData.X}$			
	Gene1	Gene2	Gene3
Spot1	0	10	24
Spot2	15	2	0
Spot3	14	12	23

Trasposta $X^\top$			
	Spot1	Spot2	Spot3
Gene1	0	15	14
Gene2	10	2	12
Gene3	24	0	23

mente vettorizzate. Prima della funzione vengono definite alcune variabili di supporto:

- **ITNI**: matrice di forma $(n_{spot}, 7)$ contenente, per ogni spot, gli indici dei suoi vicini. Gli elementi pari a -1 indicano assenza di vicino.
- **VNI** = $(\text{ITNI} \neq -1)$: maschera binaria che vale 1 dove l'indice è valido e 0 altrimenti.
- **NN** = VNI.sum(axis=1) : numero di vicini validi per ciascuno spot.
- **NM** = $(\text{VNI.T} / \text{NN.T}).\text{T}$: matrice di pesi normalizzati, dove ogni vicino valido riceve un peso pari a $\frac{1}{NN_i}$.

La funzione calcola la media dei valori di espressione di un gene sui vicini di ciascuno spot. Ogni riga della matrice `gene_data[ITNI]` contiene i valori del gene nei vicini di uno spot, che vengono moltiplicati per i pesi corrispondenti in **NM** e poi sommati. Il risultato è un vettore in cui ogni elemento rappresenta la media dei valori di espressione nei vicini.

La terza funzione, `opt_mean_filter_iterated(gene_data, it)`, semplicemente itera `it` volte la funzione precedente. Questo perchè iterando più volte il filtro di media, ottengo una rappresentazione più accurata del valore dei vicini di ogni gene, e quindi ottenendo una rappresentazione migliore delle zone più dense.

Una volta ottenuto i vari geni con le loro espressioni geniche “smussate” dal filtro di media, stimiamo per ogni gene un modello di misura $\forall k$ tale che $1 \leq k \leq 10$ gaussiane, attraverso la funzione di `sklearn.mixture.GaussianMixture()`.

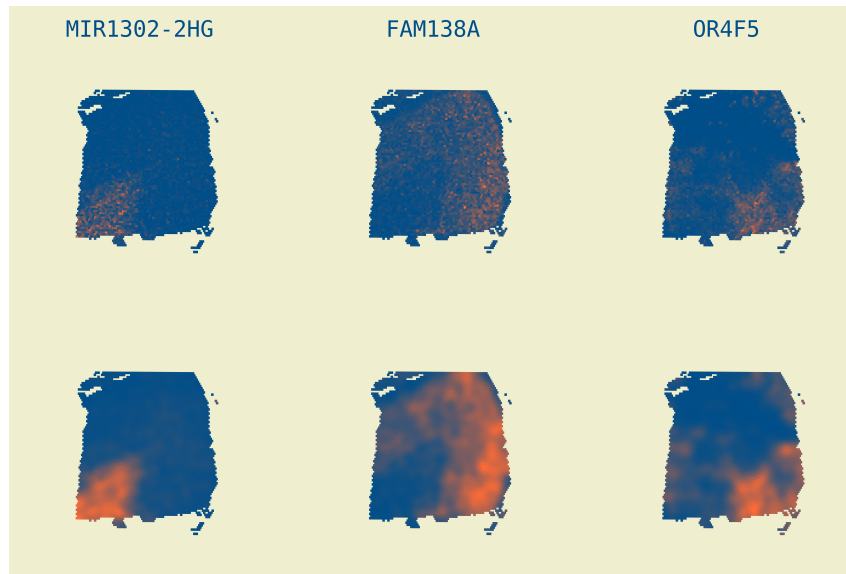


Figura 3.5: Rappresentazione di alcuni geni prima e dopo il filtro di media

Questo algoritmo cerca di adattare, per ciascun valore di k , una combinazione⁷ di k distribuzioni gaussiane ai dati di espressione del gene in esame, al fine di modellare la presenza di sottopopolazioni o modalità multiple nell'espressione.

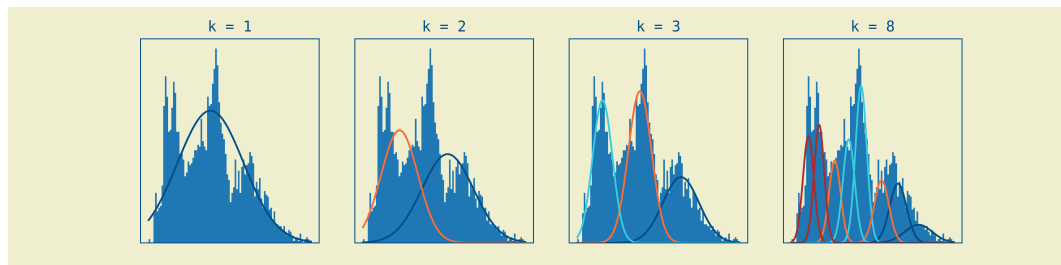


Figura 3.6: Dimostrazione di come diverse k rappresentano diversamente i dati

Si può osservare dalla figura 3.6 come con $1 \leq k$ la rappresentazione non colga efficacemente ogni picco dell'espressione genica del gene preso in esame; Si verifica perciò un fenomeno di *underfitting*⁸. Al contrario, con $k=8$ si verifica il problema opposto, ovvero ho troppe gaussiane per rappresentare efficacemente i dati in oggetto, si verifica perciò dell'*overfitting*. Nell'esempio della figura 3.6 perciò la k corretta corrisponde a $k=3$.

⁷In inglese: mixture

⁸Modello troppo semplice per catturare la complessità dei dati

Per poter scegliere in autonomia la k corretta, una volta allenata la funzione `sklearn.mixture.GaussianMixture()` con un gene, si salva attraverso la funzione `GaussianMixture().bic()` il *Bayesian Information Criterion* (BIC) corrispondente a tale k su quel gene. Il BIC è un indice statistico che misura quanto bene un modello spiega i dati passati, penalizzando i modelli troppo complessi.

La formula con la quale si può ottenere è la seguente:

$$\text{BIC} = k \cdot \ln(n) - 2 \cdot \ln(\hat{L}) \quad (3.15)$$

dove n indica il numero degli spots, k il numero di parametri stimati dal modello, $\hat{L}$ massima verosimiglianza del modello, ovvero quanto bene il modello si adatta ai dati.

Una volta ottenuti i vari BIC per ogni k , viene infine scelta la k corretta attraverso il calcolo del “Knee Point”. Il Knee Point, altro non è che il punto in cui la curva cambia inclinazione in modo evidente, passando da una salita marcata a una più piatta, o viceversa.

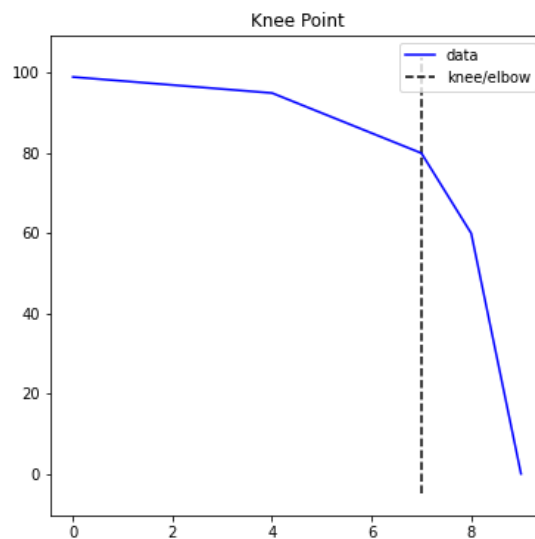


Figura 3.7: Esempio di Knee Point

Il codice Python per poter calcolare relativi BIC e Knee Point è il seguente:

```

1  def gmm_BIC(gdata, mink=2, maxk=10):
2
3      data = gdata.reshape(-1, 1)
4      bics = []

```

```

5
6     for i in range(mink, maxk+1):
7         gmm = GaussianMixture(n_components=i,
8                                random_state=0)
9                                .fit(data)
10        BIC_gmm = gmm.bic(data)
11        bics.append(BIC_gmm)
12
13    return bics
14
15    def find_knee(x, y):
16        knee_locator = kneed
17                        .KneeLocator(
18                            x,
19                            y,
20                            interp_method="polynomial",
21                            curve="convex",
22                            direction="decreasing")
23        knee = knee_locator.knee
24
25    return knee
26
27    mink = 1
28    maxk = 10
29    x = np.arange(mink, maxk+1)
30
31    bics = gmm_BIC(gdata, mink, maxk)
32    k = find_knee(x, bics, plt.gca())

```

Codice 3.3: Funzioni per ottenere BIC e KP

Una volta ottenuta la k ottimale tramite il Knee Point, si può creare la maschera binaria per il gene in esame. La maschera binaria identifica, per ogni spot, se il gene è espresso (1) oppure no (0), in base alla soglia determinata dal modello Gaussian Mixture. Il procedimento è il seguente:

1. Si allena il modello GaussianMixture con k componenti sul vettore di espressione genica smussato.
2. Si assegna ogni spot alla componente gaussiana con la media più alta, presenza del gene, oppure più bassa, assenza.
3. Si crea la maschera binaria: 1 se lo spot appartiene alla componente con media maggiore, 0 altrimenti.

Questo procedimento mi porterà ad avere un array, di lunghezza $\text{len}(n_{\text{spots}})$ in cui avrò valore 1 se il gene risulta presente nello spot s_i , oppure 0, con $1 \leq i \leq s_n$.

Il codice per generare la maschera binaria è il seguente:

```

1  def fit_MoG(gdata, k):
2      data = gdata.reshape(-1, 1)
3      gmm = GaussianMixture(n_components=k,
4                             random_state=0)
5                             .fit(data)
6
7      mus = gmm.means_
8      stds = np.sqrt(gmm.covariances_)
9      return mus, stds
10
11 def smallest_dist(mus, stds):
12     idx = np.argmin(mus)
13     return mus[idx][0], stds[idx][0][0]
14
15 def expansion_filter(gene_data):
16     global ITSNI, HNN
17     active_neigh = gene_data[ITSNI].sum(axis=1)
18     return (active_neigh >= 3)
19           .astype(int) *
20           gene_data +
21           (active_neigh > 3)
22           .astype(int) *
23           (1-gene_data)
24
25 def expansion_filter_iterated(gene_data, it):
26     for _ in range(it):
27         gene_data = expansion_filter(gene_data)
28     return gene_data
29
30 ITSNI = ITNI[:,1:]
31 HNN = NN//2
32 mus, stds = fit_MoG(gdata, k)
33 m, v = smallest_dist(mus, stds)
34 pgdata = expansion_filter_iterated((gdata>(m+1.5*v))
35                                     .astype(int), 10)

```

Codice 3.4: Funzioni per ottenere le maschere binarie una volta ottenuta la k del Knee Point

Segue la spiegazione di quanto avviene nel codice 3.4:

- La funzione `fit_MoG()` riceve il vettore di espressione smussata `gdata` e il numero di componenti k trovate dal Knee Point. Viene poi trasformata in colonna da `gdata.reshape(-1,1)` per adattarsi al modello Gaussian-

Mixture. Il modello viene addestrato e restituisce i vettori di medie `mus` e deviazioni standard `stds` delle gaussiane.

- `smallest_dist` seleziona la gaussiana con la media più bassa, salvandola nell'indice `idx`, assumendo che quella rappresenti l'assenza di espressione. Restituisce poi la media m e la deviazione standard v di quella componente.
- Nella codice `gdata > (m + 1.5 * v)).astype(int)` passato come argomento alla funzione `expansion_filter_iterated` si applica la soglia $m + 1.5 \cdot v$ per trasformare i valori continui in una prima maschera binaria grezza: gli spot con espressione maggiore rispetto a quella soglia sono considerati attivi, 1, altrimenti inattivi, 0.
- `expansion_filter` raffina questa maschera binaria guardando i vicini:
 - Si calcola il numero di vicini attivi per ciascun spot (`active_neigh`).
 - Se un vicino ha almeno 3 vicini attivi, si mantiene il valore attuale (`* gene_data`).
 - Se ha più di 3 vicini attivi, si può invertire il valore dello spot (`1 - gene_data`) per favorire l'espansione del segnale.
- `expansion_filter_iterated` applica iterativamente il filtro per `it` cicli, permettendo al pattern attivo di “espandersi” su regioni contigue.
- Infine, la variabile `pgdata` contiene la maschera binaria finale raffinata: si parte da una soglia basata sulla componente con media minima, poi si enfatizza la coerenza spaziale tramite l'espansione iterata 10 volte.



Figura 3.8: Gene da dati iniziali a maschera binaria

Capitolo 4

Sviluppo e risultati sperimentali

Dopo aver presentato nel Capitolo precedente le nozioni fondamentali e le operazioni di pre-processing necessarie alla gestione dei dati, in questo capitolo verrà descritto lo sviluppo pratico e le analisi sperimentale svolta. L'obiettivo principale è quello di progettare e realizzare una metodologia computazionale in grado di identificare *clustering alternativi* nei dati di trascrittomica spaziale, fornendo una rappresentazione più flessibile e interpretabile dei domini genici rispetto agli approcci convenzionali.

Per fare ciò è necessario che ogni gene venga e suddiviso spazialmente in sottogeni, affinché essi si possano catalogare tra di loro e poter analizzare eventuali relazioni particolari che ci sono fra un sottogene e un altro.

4.1 Lista degli spot vicini

Come prima cosa, come mostrato nella figura 3.2, è necessario individuare per ogni spot s_i l'insieme dei suoi spot adiacenti, ovvero la lista dei *vicini*. Questa informazione è fondamentale poiché consente di stabilire quando due geni si trovano in prossimità spaziale, condizione necessaria per poter poi definire relazioni di vicinanza, contiguità e sovrapposizione tra geni attivi.

I dati forniti dalla tecnologia 10x Visium contengono, per ciascuno spot, le sue coordinate sulla griglia di cattura. Tuttavia, tali coordinate sono organizzate in forma tabellare, come visto nella tabella 3.1, all'interno della struttura `sdata.obs`, e perciò non immediatamente interpretabili in termini spaziali. Per poter costruire un grafo dei vicini è quindi necessario effettuare una conversione preliminare di queste informazioni, trasformando le coordinate della griglia in coordinate cartesiane. In questo modo è possibile rappresentare ogni spot come un punto nel piano, preservando le distanze e la disposizione esagonale della griglia Visium.

```

1 index_to_cart = np.zeros((len(scddata.obs), 2))
2 max_row      = scddata.obs['array_row'].max()
3 for i, ind in enumerate(scddata.obs.index):
4     col = scddata.obs['array_col'][ind]
5     row = scddata.obs['array_row'][ind]
6     index_to_cart[i, 0] = col * (np.sqrt(3)/2)
7     index_to_cart[i, 1] = (max_row - row) * (3/2)

```

Codice 4.1: Trasformazione delle coordinate spaziali in coordinate cartesiane

Il frammento di codice 4.1 mostra come vengono calcolate e salvate le coordinate cartesiane di ciascuno spot, partendo dalle colonne `array_row` e `array_col` della tabella di input. Viene inoltre applicata una correzione di scala per tenere conto della disposizione esagonale della griglia, in cui le colonne sono sfalsate e le distanze orizzontali e verticali non sono equivalenti.

Costruzione del grafo dei vicini con k-NN. Definite le coordinate cartesiane $\mathbb{R}^2$ per ciascuno spot, il passo successivo consiste nell'identificare, per ogni spot s_i i sei spot più prossimi nella griglia esagonale 10x Visium. A tal fine è stato impiegato l'algoritmo dei *Nearest Neighbors* (k-NN), nella sua implementazione fornita dalla libreria `sklearn.neighbors`, libreria che implementa la funzione `NearestNeighbors(...)`.

L'idea dietro la funzione è la seguente: dato l'insieme di punti $\{x_1, \dots, x_n\} \in \mathbb{R}^2$, per ciascun punto $\mathbf{x}_i$ si ricercano i k vicini più prossimi secondo una metrica prefissata, come la distanza euclidea. Poiché la funzione restituisce anche il punto stesso come primo vicino, si imposta $k = 1 + 6$ così da ottenere, oltre a s_i , i sei vicini geometrici desiderati. Una volta trovati i vicini occorre applicare un filtro sulle distanze, in quanto la funzione ritorna vicini che possono avere distanze diverse tra loro, e quindi non direttamente solo i vicini prossimi allo spot. Applicando un filtro sulle distanze il nuovo risultato sarà una lista dove per ogni spot avremo i suoi vicini più prossimi.

Il codice per la creazione della lista dei vicini è come segue:

```

1 coords = index_to_cart
2 k = 1 + 6
3 nbrs = NearestNeighbors(n_neighbors=k,
4                         metric='euclidean')
5     .fit(coords)
6 dists, inds = nbrs.kneighbors(coords)
7
8 R = 1.74 #Raggio
9
10 vicini_filtrati = []

```

```

11     distanze_filtrate = []
12
13     for i in range(len(coords)):
14         di = dists[i, 1:]
15         ii = inds[i, 1:]
16         mask = di <= R #Filtro
17         vicini_filtrati.append(ii[mask])
18         distanze_filtrate.append(di[mask])

```

Codice 4.2: Creazione della lista dei vicini per ogni spot

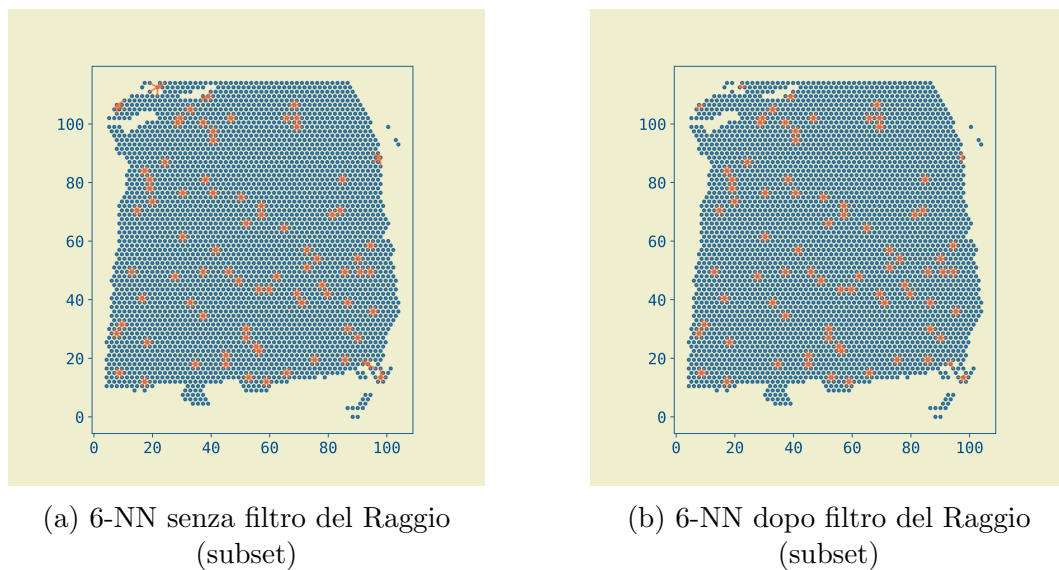


Figura 4.1: Rappresentazione grafica degli spot vicini

Dalla figura 4.1a si può vedere come senza applicare il filtro sul raggio R vengano presi come vicini anche spot diversi da quelli immediatamente prossimi allo spot in oggetto. Si può notare inoltre come non tutti gli spot possiedono 6 vicini, ma quelli in prossimità del bordo, o di buchi nel tessuto ne possiedono un quantitativo minore.

4.2 Divisione in sotto geni

Una volta trovati i vicini di ogni spot, il passo successivo consiste nel suddividere ciascun gene in sottogruppi più piccoli, detti *sottogeni*. Questa operazione è necessaria poiché l'espressione genica spaziale di un gene non si manifesta quasi mai come un unico dominio compatto, ma tende piuttosto a presentare

molteplici regioni locali di attivazione, separate tra loro da aree di non espressione. In altri termini, un singolo gene può dar luogo a più *cluster* distinti di espressione all'interno del tessuto.

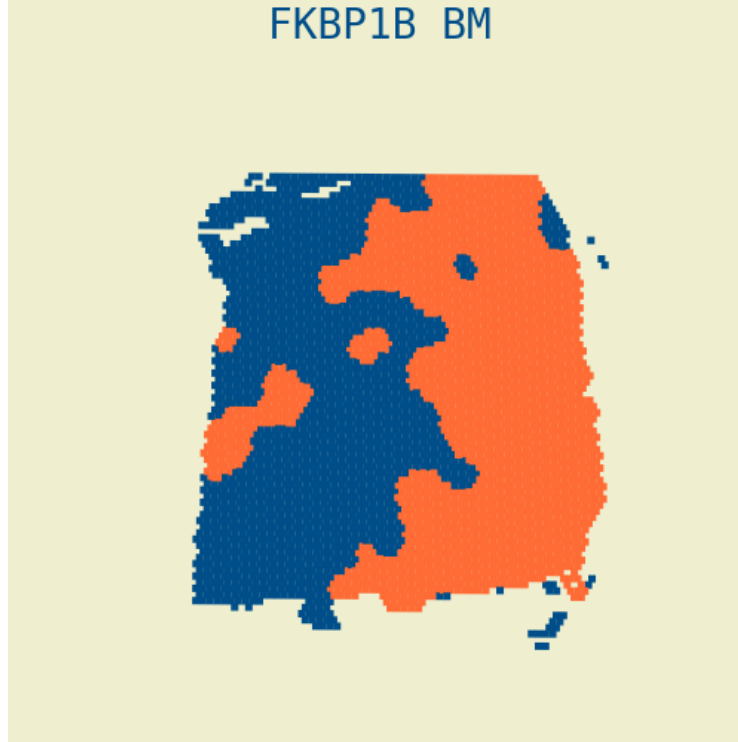


Figura 4.2: Maschera binaria di un gene con evidenti sottogeni

Come illustrato nella figura 4.2 il gene analizzato risulta suddivisibile in 3 sottogeni distinti, ciascuno dei quali corrisponde a un agglomerato spaziale di spot in cui il gene risulta espresso. Il numero totale di sottogeni dipende dalla distribuzione dell'espressione: piccoli cluster isolati al di sotto di una soglia minima vengono scartati per evitare classificazioni non significative o influenzate dal rumore tecnico. In tal modo, si ottiene una rappresentazione più pulita e interpretabile della distribuzione spaziale dell'attività genica. In altre parole per ogni gene g_j avremo h sottogeni g' con $1 \leq h$, quindi $g_j = \{g'_1, \dots, g'_h\}$.

Motivazione La suddivisione in sottogeni è una fase cruciale, poiché le successive analisi di relazione tra geni richiedono di confrontare non più le espressioni globali del gene, ma le singole regioni locali di attivazione. Ogni sottogene può infatti essere in prossimità o in relazione con un altro sottogene appartenente a un gene diverso, anche se i due geni nel loro complesso non mostrano

un pattern di co-espressione globale. Questa granularità permette di cogliere fenomeni locali che andrebbero persi in un’analisi unica.

Approccio algoritmico Per identificare automaticamente i sottogeni di ciascun gene, è stato adottato un approccio basato sulla ricerca delle *componenti connesse* nella maschera binaria del gene. Si consideri, per un dato gene, la maschera booleana $\mathbf{y} \in \{0,1\}^n$ che indica, per ciascuno spot, se il gene risulta espresso (1) o meno (0). Utilizzando la lista dei vicini precedentemente costruita, è possibile determinare quali spot attivi (con $\mathbf{y}_i = 1$) siano connessi tra loro, ovvero appartengano alla stessa regione contigua di espressione. Ogni insieme di spot tra loro connessi rappresenta una componente del grafo, e quindi un sottogene.

La ricerca delle componenti connesse viene implementata tramite l’algoritmo *Breadth-First Search* (BFS), una procedura classica di esplorazione su grafi. A partire da uno spot attivo non ancora visitato, la BFS esplora ricorsivamente tutti i suoi vicini anch’essi attivi, poi i vicini dei vicini, e così via, fino a esaurire la regione connessa. In tal modo, ogni esplorazione restituisce una componente distinta del grafo dei nodi attivi.

La funzione riportata di seguito implementa l’intera procedura descritta. Essa riceve in input la maschera binaria del gene (`mask`), la lista dei vicini per ogni spot (`neighbors`), e il numero totale di spot (`number_of_spots`). Restituisce l’elenco delle componenti connesse, le relative etichette e, se richiesto, le Binary Mask corrispondenti.

Il codice è il seguente:

```

1 def connected_components_on_mask(mask,
2                                   neighbors,
3                                   number_of_spots,
4                                   min_size=1,
5                                   return_masks=True):
6
7     mask = np.asarray(mask).astype(bool)
8     n = len(mask)
9     visited = np.zeros(n, dtype=bool)
10    comp_labels = -1 * np.ones(n, dtype=int)
11    components = []
12    comp_masks = [] if return_masks else None
13    cid = 0
14
15    # Consideriamo solo i nodi attivi
16    active_nodes = np.flatnonzero(mask)
17
18    for start in active_nodes:
```

```
19     if visited[start]:
20         continue
21     if not mask[start]:
22         continue
23
24     q = deque([start])
25     visited[start] = True
26     comp = []
27
28     while q:
29         u = q.popleft()
30         comp.append(u)
31         # espandi solo verso vicini attivi
32         for v in neighbors[u]:
33             if v < 0 or v >= n:
34                 continue
35             if mask[v] and not visited[v]:
36                 visited[v] = True
37                 q.append(v)
38
39         # Applica soglia per la dimensione componente
40         if len(comp) >= min_size:
41             for u in comp:
42                 comp_labels[u] = cid
43             components.append(comp)
44             cid += 1
45
46             if return_masks:
47                 comp_masks.append(
48                     component_to_mask(
49                         comp, number_of_spots))
50
51     if return_masks:
52         return comp_labels, components, comp_masks
53     else:
54         return comp_labels, components
```

Codice 4.3: Identificazione sottogeni tramite BFS

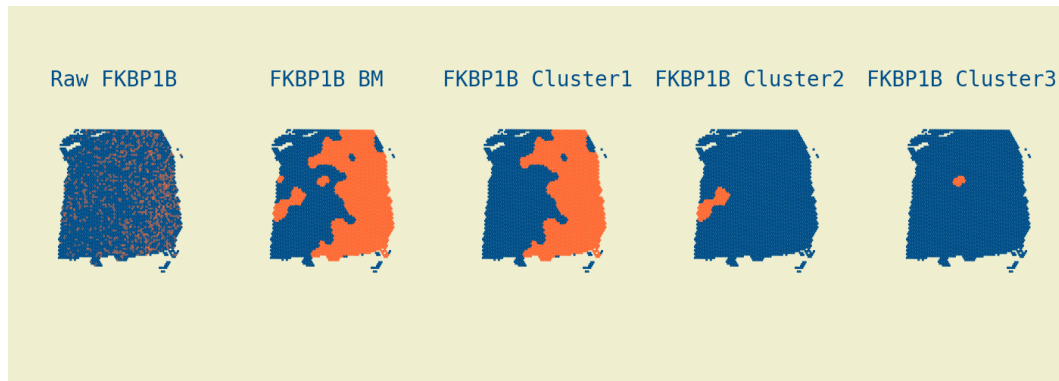


Figura 4.3: Risultato individuazione sottogeni

La figura 4.3 mostra come dato un gene vengano individuati i suoi corrispondenti sottogeni. Per poter applicare questa funzione automaticamente su tutti i geni che prendiamo in considerazione, richiamiamo la funzione all'interno di un ciclo `for()` e ci salviamo i risultati in un dictionary avente forma `['gene_id', 'sub_id', 'count', 'comp_mask']`.

```

1 def build_subgenes(binary_masks,
2                     neighbors,
3                     number_of_spots,
4                     min_size=1,
5                     gene_ids=None):
6
7     if isinstance(binary_masks, np.ndarray) and
8         binary_masks.ndim == 2:
9         n_genes = binary_masks.shape[0]
10        get_mask = lambda i: binary_masks[i]
11    else:
12        n_genes = len(binary_masks)
13        get_mask = lambda i: np.asarray(binary_masks[i])
14
15    if gene_ids is None:
16        gene_ids = list(range(n_genes))
17
18    results = []
19    for gi in range(n_genes):
20        mask = get_mask(gi)
21        _, comps, comp_masks =
22            connected_components_on_mask(
23                mask,
24                neighbors,
25                number_of_spots,
26                min_size=min_size,

```

```

27         return_masks=True
28     )
29
30     for sub_id, (comp, cmask) in
31         enumerate(zip(comps, comp_masks)):
32         results.append({
33             "gene_id": gene_ids[gi],
34             "sub_id": sub_id,
35             "count": len(comp),
36             "comp_mask": cmask
37         })
38     return results

```

Codice 4.4: Creazione completa sottogeni

4.3 Calcolo del rapporto

Ottenuta la scomposizione di ciascun gene in sottogeni, il passo successivo consiste nel determinare, per ogni coppia di sottogeni appartenenti a famiglie differenti, il *rapporto spaziale* che li lega. L'obiettivo è distinguere i casi in cui due sottogeni sono completamente separati dai casi in cui esiste una forma di contatto, sovrapposizione o contenimento geografico all'interno del tessuto.

Dato un sottogene A e un sottogene B , il rapporto viene classificato nelle seguenti categorie mutuamente esclusive:

- **DISGIUNTI**: i due sottogeni non condividono alcuno spot.
- **IN-COSTA**: sottogeni parzialmente sovrapposti senza contenimento.
- **A-CONTENUTO**: il sottogene A è contenuto in B .
- **B-CONTENUTO**: simmetricamente, B è contenuto in A .
- **VICINI**: piccola sovrapposizione trascurabile e presenza di contatto tramite la lista dei vicini: esistono spot di A che hanno, tra i loro vicini, spot appartenenti a B .

Operativamente, si rappresentano i sottogeni come insiemi di indici di spot attivi, $A, B \subseteq \{1, \dots, n\}$, e si calcolano: l'intersezione $A \cap B$ e le quote di overlap

$$\text{overlap}_A = \frac{|A \cap B|}{|A|}, \quad \text{overlap}_B = \frac{|A \cap B|}{|B|}.$$

Il *contenimento* è valutato tramite una tolleranza $\text{contain_tol} \in [0, 1]$ che ammette piccole discrepanze: A è contenuto in B se $\frac{|A \cap B|}{|A|} \leq \text{contain_tol}$, e

viceversa per B in A . La *vicinanza* è invece definita usando la lista dei vicini: A e B sono VICINI se un numero sufficiente di spot in A possiede almeno un vicino in B .

Confrontiamo solo coppie che appartengono a famiglie diverse. Il numero di confronti è quindi pari a z tale che $z \leq \sum_{i=1}^n i$ dove n sono il numero dei sottogeni totali, ossia tutte le coppie possibili meno quelle intra-famiglia.

Il codice seguente riporta la classificazione implementata. Le maschere binarie sono convertite in insiemi di indici (`mask_to_set`). La funzione `are_touching` valuta la vicinanza su grafo contando quanti spot di A hanno almeno un vicino in B ; un contatore supera una soglia assoluta (> 50) per confermare il contatto. La procedura `classify_relation_soft_matrix` applica nell'ordine: contenimento, overlap significativo, vicinanza, e in disgiunzione. Le soglie numeriche impiegate (e.g., `contain_tol=0.02`, `overlap=0.3` per IN-COSTA, `overlap=0.03` per VICINI, soglia di contatto > 50) riflettono una scelta conservativa: si privilegia un contenimento “quasi perfetto” e si minimizza il rischio di confondere overlap marginali con vera sovrapposizione.

```

1 def mask_to_set(mask):
2     maskArray = np.asarray(mask)
3     return set(np.flatnonzero(maskArray == 1))
4
5 def are_touching(A_set, B_set, neighbors):
6     count = 0
7     if not A_set or not B_set:
8         return False
9     if len(A_set) > len(B_set):
10        A_set, B_set = B_set, A_set
11    for i in A_set:
12        if any((j in B_set) for j in neighbors[i]):
13            count+=1
14    if(count>50):
15        return True
16    return False
17
18 def classify_relation_soft_matrix(maskA,
19                                  maskB,
20                                  neighbors,
21                                  contain_tol=0.02):
22     A = mask_to_set(maskA) #Binary mask 1
23     B = mask_to_set(maskB) #Binary mask 2
24
25
26     inter = A & B #intersezione.
27     overlapA = (len(inter) / len(A)) if A else 0.0

```

```

28     overlapB = (len(inter) / len(B)) if B else 0.0
29
30     missA = len(A - B)
31     missB = len(B - A)
32     if len(A) > 0 and (missA / len(A) <= contain_tol):
33         return ("A_CONTENUTO")
34
35     if len(B) > 0 and (missB / len(B) <= contain_tol):
36         return ("B_CONTENUTO")
37
38     if inter and (overlapA > 0.3) and (overlapB > 0.3):
39         return ("IN_COSTA")
40
41     if are_touching(A, B, neighbors) and (overlapA <
42     0.03) and (overlapB < 0.03):
43         return ("VICINI")
44
45     return ("DISGIUNTI")

```

Codice 4.5: Calcolo del rapporto dei sottogeni

Filtraggio dei risultati Una volta classificati vengono infine rimosse le coppie DISGIUNTE e si salvano i risultati in un dictionary: ['gene_id_a', 'sub_id_a', 'gene_id_b', 'sub_id_b', 'rapporto']. Dal dictionary si salvano infine, applicando un ulteriore filtro, oltre alla lista appena creata anche le liste contenenti senza IN_COSTA e con solo VICINI, per alimentare le analisi successive.

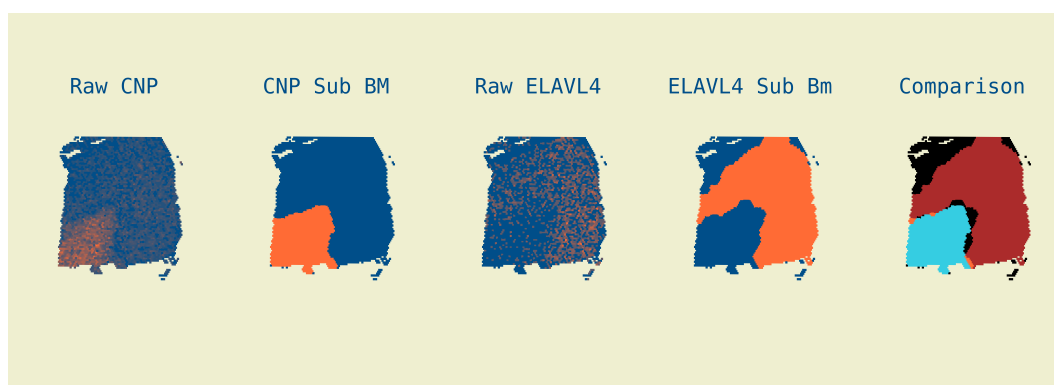


Figura 4.4: Due sottogeni VICINI messi a confronto

4.4 Grafo dei rapporti

Una volta ottenute le liste dei vari rapporti costruiamo un CVS per poter analizzare i dati appena raccolti sotto forma di grafo. Questo ci è utile per analizzare i rapporti e vederli rappresentati graficamente, in modo da scovare pattern altrimenti invisibili. Per poter creare il grafo, essendoci milioni di record diversi, si è scelto di utilizzare *Gephi* un software opensource nato appositamente per rappresentare i grafi.

il codice è il seguente:

```

1 def node_id(gene_id: int, sub_id: int) -> str:
2     return f"G_{gene_id}_S_{sub_id}"
3
4 def dump_gephi_nodes_and_edges(relations,
5                                 nodes_csv="nodes.csv",
6                                 edges_csv="edges.csv",
7                                 dedup_undirected=True,
8                                 log_every=200000):
9
10    nodes = {}
11    seen_undir = set()
12
13    with open(edges_csv, "w", encoding="utf-8") as fe:
14        we = fe.write
15        we("Source,Target,Type,Relation\n")
16        for c, r in enumerate(relations, 1):
17            ga, sa = int(r["gene_id_a"]), int(r["sub_id_a
18            "])
19            gb, sb = int(r["gene_id_b"]), int(r["sub_id_b
20            "])
21            rel = str(r["rapporto"]).upper()
22
23            # crea (o registra) i nodi
24            ida = node_id(ga, sa)
25            idb = node_id(gb, sb)
26            if ida not in nodes:
27                nodes[ida] = (ida, ga, sa)
28            if idb not in nodes:
29                nodes[idb] = (idb, gb, sb)
30
31            if rel == "A_CONTENUTO":
32                we(f"{ida},{idb},Directed,A_CONTENUTO\n")
33            elif rel == "B_CONTENUTO":
34                we(f"{idb},{ida},Directed,B_CONTENUTO\n")

```

```
33     # normalizzo il valore Relation per colore coerente
34     else:
35         if dedup_undirected:
36             key = (ida, idb) if ida <= idb else (
37                 idb, ida)
38             if key in seen_undir:
39                 continue
40             seen_undir.add(key)
41             s, t = key
42             we(f"{s},{t},Undirected,{rel}\n")
43
44             if log_every and c % log_every == 0:
45                 print(f"edges written: {c}")
46
47         with open(nodes_csv, "w", encoding="utf-8") as fn:
48             wn = fn.write
49             wn("Id,Label,Gene,Sub\n")
50             for nid, (label, g, s) in nodes.items():
51                 wn(f"{nid},{label},{g},{s}\n")
52
53     return nodes_csv, edges_csv
```

Codice 4.6: Creazione CSV grafi

Capitolo 5

Conclusione

Conclusione che riassume il lavoro svolto ed eventuali lavori futuri.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografia

- [1] Istituto dell'Enciclopedia Italiana Treccani. Informatica, n.d.
- [2] B. Pascal and C. Voza. *Pensieri*. Guaraldi, 1995.
- [3] Simon Singh and Stefano Galli. *Codici & segreti*. Rizzoli Milano, 1999.
- [4] Eric W Sayers, Jeffrey Beck, Evan E Bolton, Devon Bourexis, James R Brister, Kathi Canese, Donald C Comeau, Kathryn Funk, Sunghwan Kim, William Klimke, et al. Database resources of the national center for biotechnology information. *Nucleic acids research*, 49(D1):D10–D17, 2021.
- [5] David L. Nelson and Michael M. Cox. *I principi di biochimica di Lehninger*. Zanichelli, 2018.
- [6] Manuela Helmer Citterich, F Ferré, G Pavesi, C Romualdi, and G Pesole. *Fondamenti di bioinformatica*. Zanichelli, 2018.
- [7] National Human Genome Research Institute. Epigenomics fact sheet, 2020.
- [8] Cameron G Williams, Hyun Jae Lee, Takahiro Asatsuma, Roser Vento-Tormo, and Ashraful Haque. An introduction to spatial transcriptomics for biomedical research. *Genome medicine*, 14(1):68, 2022.
- [9] Xuanwei Chen, Qinghua Ran, Junjie Tang, Zihao Chen, Siyuan Huang, Xingjie Shi, and Ruibin Xi. Benchmarking algorithms for spatially variable gene identification in spatial transcriptomics. *Bioinformatics*, 41(4):btaf131, 03 2025.
- [10] Shun H Yip, Pak C Sham, and Jun Wang. Evaluation of tools for highly variable gene discovery from single-cell rna-seq data. *Briefings in Bioinformatics*, 20(4):1583–1589, Jul 2019.

- [11] J.K. Arora, A. Opasawatchai, S.A. Teichmann, P. Matangkasombut, and V. Charoensawan. Computational workflow for investigating highly variable genes in single-cell rna-seq across multiple time points and cell types. *STAR Protocols*, 4(3):102387, Sep 2023. Epub 2023 Jun 27.
- [12] Ruzhang Zhao, Jiuyao Lu, Weiqiang Zhou, Ni Zhao, and Hongkai Ji. A systematic evaluation of highly variable gene selection methods for single-cell rna-sequencing. *bioRxiv*, 2024.
- [13] Chao Chen, Hyun Jae Kim, and Peng Yang. Evaluating spatially variable gene detection methods for spatial transcriptomics data. *Genome Biology*, 25(1):18, 2024.
- [14] Zhen Li, Zarna M. Patel, Dong Song, Guanqun Yan, Jingjing J. Li, and Luca Pinello. Benchmarking computational methods to identify spatially variable genes and peaks. *bioRxiv*, Dec 2023. Preprint.
- [15] Yang Jin, Yuanli Zuo, Gang Li, Wenrong Liu, Yitong Pan, Ting Fan, Xin Fu, Xiaojun Yao, and Yong Peng. Advances in spatial transcriptomics and its applications in cancer research. *Molecular Cancer*, 23(1):129, Jun 2024.
- [16] B. Li, W. Zhang, C. Guo, H. Xu, L. Li, M. Fang, Y. Hu, X. Zhang, X. Yao, M. Tang, et al. Benchmarking spatial and single-cell transcriptomics integration methods for transcript distribution prediction and cell type deconvolution. *Nature Methods*, 19:662–670, 2022.
- [17] Coral Fustero-Torre, María José Jiménez-Santos, Santiago García-Martín, Carlos Carretero-Puche, Luis García-Jimeno, Vadym Ivanchuk, Tomás Di Domenico, Gonzalo Gómez-López, and Fátima Al-Shahrour. Beyond-cell: targeting cancer therapeutic heterogeneity in single-cell rna-seq data. *Genome Medicine*, 13(1):187, 2021.
- [18] Rahul Satija. Analysis, visualization, and integration of spatial datasets with Seurat — satijalab.org. https://satijalab.org/seurat/articles/spatial_vignette.html.
- [19] Y. Long, K. S. Ang, M. Li, K. L. K. Chong, R. Sethi, C. Zhong, H. Xu, Z. Ong, K. Sachaphibulkij, A. Chen, L. Zeng, H. Fu, M. Wu, L. H. K. Lim, L. Liu, and J. Chen. Spatially informed clustering, integration, and deconvolution of spatial transcriptomics with graphst. *Nature Communications*, 14(1):1155, Mar 2023.

- [20] Ruben Dries, Qian Zhu, Rui Dong, Chee-Huat Linus Eng, Huipeng Li, Kan Liu, Yuntian Fu, Tianxiao Zhao, Arpan Sarkar, Feng Bao, Rani E. George, Nico Pierson, Long Cai, and Guo-Cheng Yuan. Giotto: a tool-box for integrative analysis and visualization of spatial expression data. *Genome Biology*, 22(1):78, Mar 2021.
- [21] Gospel Ozioma Nnadi, Vincenzo Bonnici, Simone Avesani, Eva Viesi, and Rosalba Giugno. Leveraging graph information for spatially informed patient data analysis with gist. In *2025 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*, pages 1–8, 2025.
- [22] Simone Avesani, Eva Viesi, Luca Alessandrì, Giovanni Motterle, Vincenzo Bonnici, Marco Beccuti, Raffaele Calogero, and Rosalba Giugno. Stardust: improving spatial transcriptomics data analysis through space-aware modularity optimization-based clustering. *GigaScience*, 11:giac075, 08 2022.
- [23] Zhenhao Ni, Abhinav Prasad, Shanshan Chen, et al. Spotclean adjusts for spot swapping in spatial transcriptomics data. *Nature Communications*, 13:2971, 2022.
- [24] Andreas Schroeder, Maximilian L. Loth, Chao Luo, et al. Scaling up spatial transcriptomics for large-sized tissues: uncovering cellular-level tissue architecture beyond conventional platforms with iscale. *Nature Methods*, 22:1911–1922, 2025.
- [25] J Gillespie, M Pietrzak, MA Song, and D Chung. A meta-review of spatial transcriptomics analysis software. *Cells*, 14(14):1060, Jul 2025.
- [26] What is the spatial resolution and configuration of the capture area of the Visium v1 Gene Expression Slide? kb.10xgenomics.com. [Accessed 06-10-2025].
- [27] Vincenzo Bonnici and Matteo Boroni Grazioli. *Identification of Spatially Co-Expressed Genes to Support Spatial Transcriptomics*. CIBB 2025, 2025.
- [28] Tongxuan Lv, Ying Zhang, Mei Li, Qiang Kang, Shuangfang Fang, Yong Zhang, Susanne Brix, and Xun Xu. Eags: efficient and adaptive gaussian smoothing applied to high-resolved spatial transcriptomics. *GigaScience*, 13:giad097, 02 2024.

- [29] Yusong Liu, Tongxin Wang, Ben Duggan, Michael Sharpnack, Kun Huang, Jie Zhang, Xiufen Ye, and Travis S Johnson. Spcs: a spatial and pattern combined smoothing method for spatial transcriptomic expression. *Briefings in Bioinformatics*, 23(3):bbac116, 04 2022.
- [30] Tongxuan Lv, Ying Zhang, Mei Li, Qiang Kang, Shuangfang Fang, Yong Zhang, Susanne Brix, and Xun Xu. Eags: efficient and adaptive gaussian smoothing applied to high-resolved spatial transcriptomics. *GigaScience*, 13:giad097, 02 2024.
- [31] Yusong Liu, Tongxin Wang, Ben Duggan, Michael Sharpnack, Kun Huang, Jie Zhang, Xiufen Ye, and Travis S Johnson. Spcs: a spatial and pattern combined smoothing method for spatial transcriptomic expression. *Briefings in Bioinformatics*, 23(3):bbac116, 04 2022.
- [32] F. Alexander Wolf, Philipp Angerer, and Fabian J. Theis. Scanpy: large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1):15, 2018.
- [33] Isaac Virshup, Danila Bredikhin, Lukas Heumos, Giovanni Palla, Gregor Sturm, Adam Gayoso, Ilia Kats, Mikaela Koutrouli, Philipp Angerer, Volker Bergen, Pierre Boyeau, Maren Büttner, Gokcen Eraslan, David Fischer, Max Frank, Justin Hong, Michal Klein, Marius Lange, Romain Lopez, Mohammad Lotfollahi, Malte D. Luecken, Fidel Ramirez, Jeffrey Regier, Sergei Rybakov, Anna C. Schaar, Valeh Valiollah Pour Amiri, Philipp Weiler, Galen Xing, Bonnie Berger, Dana Pe’er, Aviv Regev, Sarah A. Teichmann, Francesca Finotello, F. Alexander Wolf, Nir Yosef, Oliver Stegle, Fabian J. Theis, and Scverse Community. The scverse project provides a computational ecosystem for single-cell omics data analysis. *Nature Biotechnology*, 41(5):604–606, May 2023.

